
Royaume du Maroc • Université Hassan II de Casablanca
École Nationale Supérieure d'Électricité et de Mécanique – ENSEM

Département : Génie Informatique
Filière : Génie Logiciel et Intelligence Artificielle

Logo
Université Hassan II

Logo
ENSEM

RAPPORT DE PROJET DE FIN D'ÉTUDES

Réalisé au sein de [Nom de la Société]

Logo Société

THÈME :

GRC AI Analyzer

Plateforme d'analyse automatisée des rapports
de gouvernance, risque et conformité
propulsée par l'Intelligence Artificielle Générative

Soutenu le ____ - ____ - 2026 **par :**

– Mahdi BOUKHOUALI

Encadré par :

– Prof. [Nom Prénom] (Encadrant
ENSEM)

– M. [Nom Prénom] (Encadrant In-
dustriel)

Devant les membres du jury :

– Pr. [Nom Prénom] Président

– Pr. [Nom Prénom] Rapporteur

– Pr. [Nom Prénom] Encadrant ENSEM

– M. [Nom Prénom] Encadrant Industriel

Promotion 2026

Dédicace

C'est avec une profonde gratitude et une immense fierté que je dédie ce modeste travail, fruit de plusieurs mois d'efforts et de recherche, aux personnes qui ont marqué mon parcours académique et personnel.

Je dédie ce rapport en premier lieu à ma famille, et plus particulièrement à mes chers parents, qui ont toujours été ma source d'inspiration et le pilier de ma réussite. Aucun mot ne saurait exprimer l'ampleur de ma reconnaissance pour leurs innombrables sacrifices, leur soutien inconditionnel et leurs prières qui m'ont accompagné dans chaque étape.

À mes frères et sœurs, pour leur présence réconfortante et leur belle complicité. Que ce travail soit l'humble témoignage de mon affection et de ma gratitude éternelle.

Je dédie également ce projet à mes encadrants, dont l'orientation rigoureuse, la disponibilité et les précieux conseils ont été une véritable boussole tout au long de ce stage de fin d'études.

Enfin, je souhaite dédier ce travail à tous mes amis, collègues et professeurs de l'école, qui ont contribué, de près ou de loin, à mon évolution et à l'aboutissement de ce projet.

Remerciements

Je tiens à exprimer ma plus sincère gratitude à l'ensemble des personnes qui ont contribué à la réalisation de ce travail, tant sur le plan académique que professionnel et personnel.

Tout d'abord, je remercie chaleureusement mes parents pour leur amour inconditionnel, leur patience inépuisable et leur soutien indéfectible tout au long de mon parcours. Leur confiance en moi et leurs encouragements constants ont été le pilier de ma réussite.

Je tiens également à exprimer toute ma reconnaissance à mon encadrant pédagogique, Prof. [Nom de l'encadrant académique], pour sa disponibilité, ses conseils judicieux et son accompagnement rigoureux. Son regard critique et ses orientations méthodologiques ont grandement contribué à la qualité de ce projet.

Mes remerciements vont également à mon encadrant professionnel, M. [Nom de l'encadrant industriel], pour sa confiance, son suivi attentif et son partage d'expertise dans le domaine de la gouvernance, du risque et de la conformité. Sa vision technique et métier a grandement enrichi ce travail.

Je remercie aussi l'ensemble de l'équipe avec laquelle j'ai eu le privilège de collaborer durant cette période. Les échanges enrichissants et l'esprit collaboratif ont fait de cette expérience une aventure humaine et professionnelle inoubliable.

Enfin, j'adresse mes remerciements à l'ensemble du corps professoral de l'ENSEM, ainsi qu'à mes camarades de promotion, pour les années de formation qui ont rendu possible ce projet.

Résumé

Dans un contexte mondial marqué par la multiplication des réglementations en matière de protection des données et de cybersécurité, les organisations sont confrontées à une charge croissante d'analyse de leurs rapports de gouvernance, de risque et de conformité (GRC). L'analyse manuelle de ces documents est chronophage, sujette aux erreurs et difficilement scalable.

C'est dans ce cadre que s'inscrit ce projet de fin d'études, dont l'objectif est la conception et la réalisation de **GRC AI Analyzer**, une plateforme *full-stack* d'analyse automatisée des rapports GRC propulsée par l'intelligence artificielle générative. Le système permet l'import de documents aux formats PDF, DOCX et XLSX, puis exploite le modèle **Mistral AI** pour extraire automatiquement les risques, évaluer la conformité vis-à-vis des référentiels **ISO 27001**, **RGPD** et de la **Loi 09-08** marocaine, et générer des recommandations concrètes.

L'architecture repose sur un backend **FastAPI** asynchrone, une persistance **PostgreSQL**, un stockage objet **MinIO**, une base vectorielle **ChromaDB** pour le *Retrieval Augmented Generation* (RAG), et une interface utilisateur **React/Vite** moderne. Un pipeline d'analyse en deux phases parallèles assure des temps de traitement réduits, tandis qu'une interface conversationnelle basée sur le RAG permet à l'utilisateur d'interroger directement le rapport analysé.

Mots-clés : GRC, Intelligence Artificielle, Mistral, RAG, ChromaDB, FastAPI, React, ISO 27001, RGPD, Loi 09-08, conformité, gestion des risques.

Abstract

In a global context marked by the multiplication of regulations regarding data protection and cybersecurity, organizations face a growing burden in analyzing their Governance, Risk and Compliance (GRC) reports. Manual analysis of these documents is time-consuming, error-prone and hardly scalable.

This end-of-studies project introduces **GRC AI Analyzer**, a full-stack platform for automated GRC report analysis powered by generative artificial intelligence. The system enables document upload in PDF, DOCX and XLSX formats, then leverages the **Mistral AI** model to automatically extract risks, evaluate compliance against **ISO 27001**, **GDPR** and Morocco's **Law 09-08**, and generate concrete recommendations.

The architecture relies on an asynchronous **FastAPI** backend, a **PostgreSQL** database, a **MinIO** object store, a **ChromaDB** vector database for Retrieval Augmented Generation (RAG), and a modern **React/Vite** user interface. A two-phase parallel analysis pipeline ensures reduced processing times, while a conversational interface based on RAG allows users to directly interrogate analyzed reports.

Keywords : GRC, Artificial Intelligence, Mistral, RAG, ChromaDB, FastAPI, React, ISO 27001, GDPR, Law 09-08, compliance, risk management.

Table des matières

Dédicace	I
Remerciements	II
Résumé	III
Abstract	IV
Liste des acronymes	XI
1 Contexte du projet	1
1.1 Introduction	1
1.2 Présentation de l'organisme d'accueil	1
1.2.1 Présentation générale	1
1.2.2 Domaines d'expertise et offre de services	1
1.2.3 Positionnement et clientèle	2
1.2.4 Cadre du projet GRC AI Analyzer	2
1.3 Contexte métier : la Gouvernance, le Risque et la Conformité (GRC) . . .	2
1.3.1 Les trois piliers de la GRC	3
1.3.2 Cadres réglementaires et normatifs retenus	4
1.3.3 Enjeux de l'analyse documentaire GRC	5
1.4 Étude de l'existant	5
1.5 Problématique du projet	6
1.6 Solution envisagée	6
1.7 Objectifs du projet	7
1.8 Conduite du projet	8
1.9 Synthèse du chapitre	8
2 État de l'art et étude de l'existant	9

2.1	Introduction	9
2.2	Pratiques actuelles d'analyse des rapports GRC	9
2.3	Solutions du marché et leurs limites	9
2.3.1	Leaders du marché GRC	10
2.3.2	Solutions open-source	10
2.3.3	Outils basés sur les LLM	10
2.4	Fondements technologiques retenus	11
2.4.1	Les grands modèles de langage (LLM)	11
2.4.2	Le Retrieval Augmented Generation (RAG)	11
2.4.3	Les bases vectorielles	12
2.5	Synthèse et positionnement	12
3	Analyse des besoins et conception	13
3.1	Introduction	13
3.2	Analyse des besoins	13
3.2.1	Identification des acteurs	13
3.2.2	Besoins fonctionnels	13
3.2.3	Besoins non fonctionnels	16
3.3	Diagramme des Cas d'Utilisation	17
3.3.1	Description générale	17
3.3.2	Description des cas d'utilisation principaux	19
3.4	Diagramme de Classes	20
3.4.1	Vue d'ensemble	20
3.4.2	Description des classes principales	21
3.5	Diagramme de Séquence	23
3.5.1	Séquence principale : Analyse d'un rapport GRC	23
3.5.2	Description détaillée des interactions	24
3.6	Diagramme d'Activité	26
3.6.1	Vue d'ensemble du flux de traitement	26
3.6.2	Description des activités	27

3.7	Architecture globale de la solution	28
3.7.1	Vue d'ensemble en couches	28
3.7.2	Diagramme de déploiement	30
3.8	Synthèse du chapitre	32
4	Conception technique et choix technologiques	33
4.1	Pile technologique retenue	33
4.2	Architecture en couches du backend	34
4.3	Conception du modèle de données	35
4.4	Conception des routes API	37
4.4.1	Lancement d'une analyse (HTTP 202 + BackgroundTask)	38
4.4.2	Polling du statut d'analyse (frontend)	39
4.4.3	Interface de chat RAG	39
4.5	Conception des prompts IA	40
4.5.1	Prompt d'extraction des risques	40
4.5.2	Prompts de vérification de conformité	42
4.5.3	Prompt de génération des recommandations	43
4.5.4	Prompt du résumé exécutif	45
4.6	Pipeline d'analyse IA asynchrone	46
4.6.1	Phase 1 — cinq tâches simultanées	46
4.6.2	Phase 2 — recommandations et résumé	47
4.6.3	Sérialisation anti-429 des appels Mistral	48
4.7	Parseur de documents et pipeline RAG	49
4.7.1	Extraction de texte multi-format	49
4.7.2	Indexation et recherche vectorielle ChromaDB	50
4.8	Sécurité applicative	51
4.8.1	Authentification JWT et hachage bcrypt	51
4.8.2	Validation des fichiers (magic bytes)	51
4.8.3	Rate limiting par adresse IP	52
4.9	Synthèse du chapitre	52

5	Implémentation et résultats	53
5.1	Environnement de développement	53
5.1.1	Configuration de l'environnement	53
5.1.2	Lancement du projet	53
5.2	Résultats obtenus	54
5.2.1	Authentification et tableau de bord	54
5.2.2	Analyse d'un rapport GRC	55
5.2.3	Chat RAG et export	56
5.3	Bilan technique	58
5.3.1	Métriques de performance	58
5.3.2	Couverture fonctionnelle	59
5.4	Synthèse	59
	Conclusion générale	60

Table des figures

1.1	Architecture simplifiée de la solution cible	7
1.2	Phases de développement du projet GRC AI Analyzer	8
2.1	Limites identifiées dans les pratiques actuelles	10
2.2	Architecture RAG utilisée dans GRC AI Analyzer	11
3.1	Diagramme des cas d'utilisation — GRC AI Analyzer	18
3.2	Diagramme de classes — modèle du domaine GRC AI Analyzer	21
3.3	Diagramme de séquence — Analyse d'un rapport GRC	24
3.4	Diagramme d'activité — flux de traitement GRC AI Analyzer	26
3.5	Architecture globale — GRC AI Analyzer	29
3.6	Diagramme de déploiement — GRC AI Analyzer	30
4.1	Organisation en cinq couches du backend FastAPI	34
4.2	Modèle relationnel de GRC AI Analyzer (PostgreSQL)	36
4.3	Les quatre prompts Mistral et leurs sorties	40
4.4	Pipeline d'analyse IA en deux phases parallèles	46
5.1	Page de connexion de GRC AI Analyzer	54
5.2	Tableau de bord principal	55
5.3	Interface de téléchargement d'un rapport	55
5.4	Matrice des risques extraits par l'IA	56
5.5	Radar de conformité multi-référentiel	56
5.6	Chat RAG contextuel sur le rapport analysé	57
5.7	Export des résultats en PDF et Excel	57
5.8	Panneau d'administration	58

Liste des tableaux

1.1	Référentiels couverts par la plateforme	4
2.1	Comparatif des solutions GRC existantes	11
4.1	Pile technologique de GRC AI Analyzer	33
4.2	Pile technologique de GRC AI Analyzer	34
4.3	Routes API principales de GRC AI Analyzer	37
5.1	Environnement de développement	53
5.2	Métriques de performance observées	58
5.3	Couverture fonctionnelle de GRC AI Analyzer	59

Liste des acronymes

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
GRC	Governance, Risk and Compliance
ISO	International Organization for Standardization
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object Relational Mapping
PDF	Portable Document Format
PFE	Projet de Fin d'Études
RAG	Retrieval Augmented Generation
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UX	User Experience

1. Contexte du projet

1.1 Introduction

Ce premier chapitre a pour objectif de placer le projet dans son environnement professionnel, métier et réglementaire. Avant d’aborder l’analyse détaillée des besoins et la conception de la solution, il est nécessaire de comprendre le contexte dans lequel s’inscrit la création de *GRC AI Analyzer*, une plateforme d’analyse automatisée des rapports de gouvernance, risque et conformité propulsée par l’intelligence artificielle.

Le chapitre présente d’abord l’organisme d’accueil, ainsi que le contexte métier de la GRC. Il expose ensuite les enjeux réglementaires (ISO 27001, RGPD, Loi 09-08), formule la problématique du projet et présente la solution et les objectifs envisagés.

1.2 Présentation de l’organisme d’accueil

1.2.1 Présentation générale

Evolvity Partners est un cabinet marocain de conseil spécialisé en cybersécurité et en gestion des risques, fondé en 2022. S’appuyant sur un pool d’experts capitalisant plus de douze années d’expérience cumulée, la structure a connu une croissance rapide : deux consultants à sa création, quatre en 2024, dix en 2025, avec une projection d’expansion internationale à l’horizon 2027. Ses locaux accueillent aujourd’hui une équipe pluridisciplinaire dédiée à accompagner ses clients au Maroc et à l’international, selon une approche sur mesure conforme aux standards internationaux les plus exigeants.

1.2.2 Domaines d’expertise et offre de services

L’offre d’Evolvity Partners est structurée autour de quatre pôles complémentaires couvrant l’ensemble du spectre de la cybersécurité.

- **Audit** : Le pôle Audit déploie des missions d’évaluation à plusieurs niveaux : Organisationnel, Technique et Physique.
- **GRC** : Le pôle GRC constitue le cœur stratégique de l’activité et couvre :
- **Technologie** : Le pôle Technologie couvre les domaines émergents : Cloud, Blockchain, IoT et OT, avec une capacité d’intégration de solutions sécurisées dans des

environnements complexes et hétérogènes.

- **ISS As A Service** : Ce modèle de prestation externalisée met à disposition des clients des profils spécialisés en mode régie ou forfait : RSSI (*Responsable de la Sécurité des Systèmes d'Information*), SecOps, PMO et experts métier, offrant ainsi une flexibilité adaptée aux organisations ne disposant pas de ces compétences en interne.

1.2.3 Positionnement et clientèle

Evolvity Partners s'adresse principalement aux banques, institutions financières, administrations publiques et grandes entreprises opérant dans des secteurs réglementés. Son positionnement repose sur une approche qui allie rigueur technique, conformité réglementaire et gestion des risques stratégiques, avec pour objectif de protéger les actifs numériques de ses clients tout en accompagnant leur transformation digitale.

1.2.4 Cadre du projet GRC AI Analyzer

Le projet *GRC AI Analyzer*, objet du présent rapport, s'inscrit directement dans le pôle GRC d'Evolvity Partners. Face au volume croissant de rapports d'audit, de politiques de sécurité et de documents de conformité traités par les consultants, l'entreprise a identifié un besoin stratégique d'automatisation de l'analyse documentaire. Le projet vise à industrialiser cette pratique grâce à une plateforme propulsée par l'intelligence artificielle générative, réduisant ainsi les délais d'analyse et augmentant la précision des diagnostics de conformité.

1.3 Contexte métier : la Gouvernance, le Risque et la Conformité (GRC)

La GRC (*Governance, Risk and Compliance*) désigne le cadre intégré par lequel une organisation structure et coordonne ses pratiques de gouvernance, de gestion des risques et de conformité réglementaire. Ces trois dimensions forment un système cohérent : une gouvernance solide oriente la gestion des risques, qui elle-même conditionne la capacité de l'organisation à satisfaire ses obligations de conformité. Dans un contexte où les cybermenaces se multiplient et où les exigences réglementaires s'intensifient, la GRC est devenue un levier stratégique indispensable pour toute organisation souhaitant maîtriser son exposition et préserver la confiance de ses parties prenantes.

1.3.1 Les trois piliers de la GRC

Gouvernance

La gouvernance définit les orientations stratégiques et les responsabilités en matière de sécurité de l'information. Elle comprend :

- les politiques et chartes de sécurité ;
- les rôles clés (RSSI, DPO, comités) ;
- le suivi via des indicateurs de performance ;
- la sensibilisation et la formation des collaborateurs.

Elle constitue la base de la gestion des risques et de la conformité.

Gestion des risques

La gestion des risques consiste à identifier, analyser et traiter les menaces pouvant affecter l'organisation. Les principales étapes sont :

1. identification des actifs et menaces ;
2. analyse de l'impact et de la probabilité ;
3. priorisation des risques ;
4. traitement : réduction, acceptation, transfert ou évitement ;
5. suivi continu.

Les références courantes incluent ISO 31000, ISO 27005 et EBIOS Risk Manager.

Conformité

La conformité garantit le respect des lois, normes et obligations applicables. Elle repose sur :

- une veille réglementaire ;
- des audits réguliers ;
- des actions correctives ;
- la production de preuves et rapports.

Une bonne conformité limite les risques juridiques, financiers et réputationnels.

1.3.2 Cadres réglementaires et normatifs retenus

Le projet GRC AI Analyzer s'appuie sur trois référentiels complémentaires couvrant la sécurité de l'information et la protection des données au niveau international et marocain.

Référentiel	Domaine	Portée
ISO 27001 :2022	Sécurité de l'information	Internationale
RGPD	Protection des données personnelles	Union Européenne
Loi 09-08	Protection des données personnelles	Maroc

TABLE 1.1 – Référentiels couverts par la plateforme

ISO 27001 :2022

ISO 27001 est la norme internationale de gestion de la sécurité de l'information. Elle définit les exigences d'un SMSI (*Système de Management de la Sécurité de l'Information*) et propose 93 contrôles de sécurité organisationnels, humains, physiques et technologiques. La plateforme utilise ce référentiel pour réaliser des analyses d'écarts (*gap analysis*) et évaluer le niveau de conformité des organisations.

RGPD

Le RGPD est le règlement européen sur la protection des données personnelles, applicable à toute organisation traitant des données de citoyens européens. Il repose sur plusieurs principes essentiels :

- la licéité et la transparence des traitements ;
- la limitation des finalités ;
- la minimisation des données ;
- le respect des droits des personnes ;
- l'obligation de prouver la conformité.

Le non-respect du RGPD peut entraîner des sanctions financières importantes.

Loi 09-08

La Loi 09-08 est le cadre marocain de protection des données personnelles, supervisé par la CNDP. Elle impose :

- la déclaration des traitements de données ;

- le respect des droits des personnes concernées ;
- la sécurisation et la confidentialité des données ;
- le contrôle des transferts de données à l'étranger.

Dans le contexte marocain, les entreprises doivent souvent respecter à la fois le RGPD et la Loi 09-08.

1.3.3 Enjeux de l'analyse documentaire GRC

Les cabinets de conseil en GRC produisent et analysent quotidiennement un volume considérable de documents : politiques de sécurité, rapports d'audit, plans de traitement des risques, preuves de conformité. L'analyse manuelle de ces documents est chronophage, sujette aux erreurs d'interprétation et difficile à standardiser entre consultants. C'est précisément ce verrou opérationnel que le projet GRC AI Analyzer vise à lever, en automatisant l'extraction, la structuration et l'évaluation des informations pertinentes contenues dans ces documents, au regard des exigences des référentiels ISO 27001, RGPD et Loi 09-08.

1.4 Étude de l'existant

L'analyse documentaire GRC repose aujourd'hui sur des processus manuels chronophages et sujets aux erreurs. Les solutions commerciales (RSA Archer, ServiceNow GRC, MetricStream) et open-source (Eramba, SimpleRisk) couvrent la gestion des risques mais ne proposent pas d'analyse automatique de documents non structurés. L'état de l'art complet, les pratiques actuelles et le positionnement de GRC AI Analyzer par rapport à ces solutions sont détaillés au chapitre 2.

Outils basés sur les LLM

L'intégration des modèles de langage dans le domaine GRC est embryonnaire. Quelques expérimentations existent : usage de ChatGPT en mode ad hoc pour résumer des politiques, ou plugins Copilot pour l'analyse de documents Office. Ces usages restent non structurés, sans ancrage aux référentiels normatifs, sans traçabilité des sources, et inadaptés à un contexte professionnel d'audit où la précision et la justification des écarts sont indispensables.

Synthèse. Aucune solution existante ne combine simultanément une analyse automatisée de documents non structurés, une couverture multi-référentiels incluant la Loi 09-08, et une interface conversationnelle ancrée aux normes GRC. Cette lacune justifie la conception de GRC AI Analyzer.

1.5 Problématique du projet

L'analyse des rapports GRC présente une dualité trompeuse, illustrée par la métaphore de l'iceberg : ce qui est visible — le rapport rendu et la conformité déclarée, les recommandations transmises au client — ne représente qu'une infime partie du travail réel. La masse immergée, invisible au commanditaire, concentre l'essentiel de l'effort et des risques : des dizaines d'heures d'analyse manuelle, d'annotations dispersées et de relectures répétées ; une interprétation des normes qui varie selon l'expert, sans traçabilité des choix effectués ; et surtout, des risques non détectés et des priorités fixées à l'intuition, faute de temps ou de méthode. C'est précisément cette partie invisible qui constitue le terreau des non-conformités non identifiées et des recommandations incomplètes.

La problématique centrale peut dès lors être formulée comme suit :

Comment concevoir une plateforme intelligente capable d'automatiser l'analyse des rapports GRC hétérogènes (PDF, DOCX, XLSX), d'extraire les risques, d'évaluer la conformité vis-à-vis des référentiels ISO 27001, RGPD et Loi 09-08, et de fournir une interface conversationnelle exploitant le contenu analysé, tout en garantissant performance, fiabilité et sécurité ?

1.6 Solution envisagée

La solution proposée, baptisée **GRC AI Analyzer**, est une plateforme *full-stack* composée d'un module d'ingestion documentaire prenant en charge PDF, DOCX et XLSX, d'un pipeline d'analyse asynchrone mobilisant le LLM Mistral AI, d'un moteur de Retrieval Augmented Generation pour le chat conversationnel, et d'une interface utilisateur React pour la visualisation et l'export des résultats.

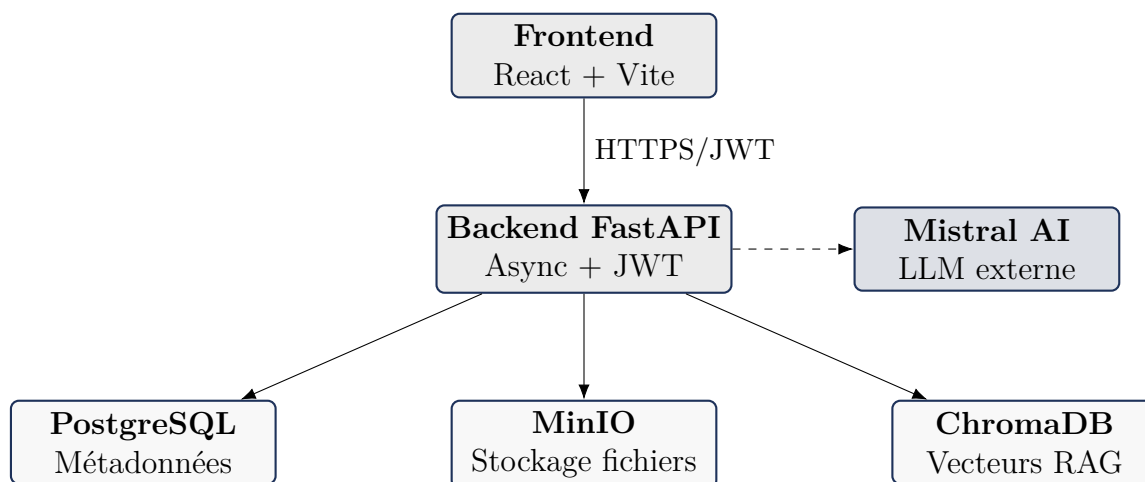


FIGURE 1.1 – Architecture simplifiée de la solution cible

1.7 Objectifs du projet

Le projet GRC AI Analyzer poursuit plusieurs objectifs visant à automatiser et faciliter l'analyse documentaire GRC.

1. Automatiser l'extraction des risques.

La plateforme doit analyser des documents (PDF, DOCX, XLSX) afin d'identifier automatiquement les risques et de réduire le temps d'analyse.

2. Évaluer la conformité multi-référentiels.

La solution doit détecter les écarts de conformité par rapport à ISO 27001, au RGPD et à la Loi 09-08, avec une traçabilité claire des contrôles concernés.

3. Générer des recommandations contextuelles.

La plateforme doit proposer des recommandations adaptées aux risques identifiés et priorisées selon leur criticité.

4. Fournir une interface conversationnelle (RAG).

Un module RAG doit permettre d'interroger les rapports en langage naturel pour obtenir des explications, synthèses ou informations ciblées.

5. Permettre l'export des résultats.

Les analyses doivent pouvoir être exportées sous forme de rapports PDF et de tableaux de bord Excel exploitables en contexte professionnel.

Convergence des objectifs. Ces cinq objectifs forment un pipeline cohérent : l'extraction alimente l'évaluation, qui fonde les recommandations, que l'interface conversationnelle rend explorable, et que l'export matérialise en livrables. Chaque composant de la plateforme correspond directement à l'un de ces objectifs.

1.8 Conduite du projet

La conduite du projet s'appuie sur une démarche itérative et incrémentale, organisée autour de quatre grandes phases de développement, depuis la mise en place du socle technique jusqu'à la livraison des fonctionnalités avancées d'export et de supervision.



FIGURE 1.2 – Phases de développement du projet GRC AI Analyzer

1.9 Synthèse du chapitre

Ce chapitre a permis de cadrer le projet dans son environnement professionnel et réglementaire. L'étude du domaine GRC a mis en évidence la nécessité d'une solution intelligente, automatisée et adaptée au contexte marocain. La problématique est clairement identifiée et les objectifs sont définis. Le chapitre suivant dresse l'état de l'art des solutions existantes et positionne GRC AI Analyzer par rapport à la littérature et au marché.

2. État de l’art et étude de l’existant

2.1 Introduction

Ce chapitre dresse l’état de l’art des technologies et solutions pertinentes pour le domaine de l’analyse GRC automatisée. Il analyse les pratiques actuelles, les limites des outils existants et les apports des modèles de langage de grande taille (LLM) et du Retrieval Augmented Generation (RAG), afin de positionner clairement la valeur ajoutée de GRC AI Analyzer.

2.2 Pratiques actuelles d’analyse des rapports GRC

L’analyse des rapports de gouvernance, risque et conformité est aujourd’hui entièrement manuelle au sein des cabinets de conseil. Ce processus mobilise un consultant senior pendant plusieurs jours et se décompose en six étapes séquentielles.

1. **Réception du rapport** : le document est transmis au cabinet sous forme de fichier PDF, Word ou scan.
2. **Lecture intégrale par un consultant senior** : un expert parcourt l’intégralité du document pour former un premier jugement.
3. **Surlignage manuel des passages à risque** : le consultant annote les passages suspects sans garantie d’exhaustivité.
4. **Saisie dans un tableur Excel ou Word** : retranscription manuelle dans un modèle de rapport structuré.
5. **Recoupement manuel avec les contrôles** : mise en correspondance avec ISO 27001, RGPD, Loi 09-08.
6. **Rédaction du rapport de synthèse** : production d’un rapport final consolidé.

2.3 Solutions du marché et leurs limites

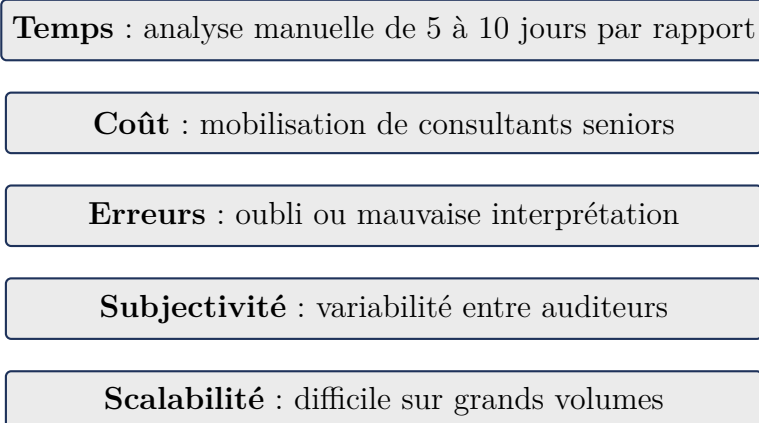


FIGURE 2.1 – Limites identifiées dans les pratiques actuelles

2.3.1 Leaders du marché GRC

Les plateformes commerciales dominantes — RSA Archer, ServiceNow GRC et MetricStream — offrent des fonctionnalités étendues de gestion des risques, des audits et de la conformité. Destinées aux grandes entreprises, elles souffrent de limitations communes : coûts de licence prohibitifs, déploiements longs et complexes, et surtout une architecture basée sur la saisie manuelle de données structurées. Aucune ne propose d’analyse automatique de documents non structurés, et aucune ne couvre nativement le cadre réglementaire marocain (Loi 09-08).

2.3.2 Solutions open-source

Des outils tels qu’Eramba, SimpleRisk ou OpenGRC proposent des alternatives accessibles. Leurs limites sont toutefois significatives : interfaces rudimentaires, absence de support des référentiels régionaux, communautés restreintes, et aucune capacité d’analyse sémantique ou d’intelligence artificielle intégrée.

2.3.3 Outils basés sur les LLM

L’intégration des modèles de langage dans le domaine GRC est embryonnaire. Quelques expérimentations existent : usage de ChatGPT en mode ad hoc pour résumer des politiques, ou plugins Copilot pour l’analyse de documents Office. Ces usages restent non structurés, sans ancrage aux référentiels normatifs, sans traçabilité des sources.

Solution	Analyse auto	Multi-réf.	Loi 09-08	RAG	Open
RSA Archer	✗	✓	✗	✗	✗
ServiceNow GRC	✗	✓	✗	✗	✗
Eramba	✗	✗	✗	✗	✓
ChatGPT (ad hoc)	✓	✗	✗	✗	✗
GRC AI Analyzer	✓	✓	✓	✓	✓

TABLE 2.1 – Comparatif des solutions GRC existantes

Synthèse. Aucune solution existante ne combine simultanément une analyse automatisée de documents non structurés, une couverture multi-référentiels incluant la Loi 09-08, et une interface conversationnelle ancrée aux normes GRC. Cette lacune justifie la conception de GRC AI Analyzer.

2.4 Fondements technologiques retenus

2.4.1 Les grands modèles de langage (LLM)

Les LLM (*Large Language Models*) sont des modèles de Deep Learning entraînés sur d'immenses corpus textuels. Ils permettent de comprendre et générer du langage naturel avec une qualité proche de celle d'un expert humain. Dans le cadre de GRC AI Analyzer, le modèle **Mistral Large** a été retenu pour ses performances sur les tâches d'extraction JSON structurée, sa compatibilité avec la langue française et sa disponibilité via une API.

2.4.2 Le Retrieval Augmented Generation (RAG)

Le RAG¹ est une architecture hybride qui combine la recherche sémantique dans une base vectorielle et la génération de texte par un LLM. Plutôt que de s'appuyer uniquement sur sa mémoire paramétrique, le modèle reçoit des extraits pertinents du document source comme contexte, ce qui réduit les hallucinations et améliore la précision des réponses.

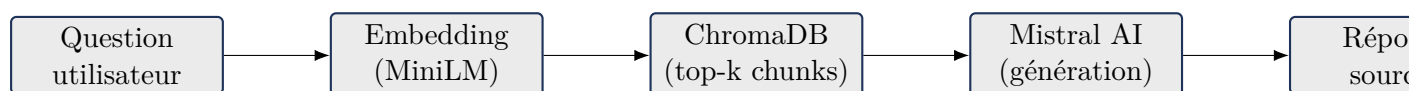


FIGURE 2.2 – Architecture RAG utilisée dans GRC AI Analyzer

1. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.

2.4.3 Les bases vectorielles

Une base vectorielle (*vector store*) indexe des représentations sémantiques denses (embeddings) de fragments textuels. La recherche par similarité cosinus permet de retrouver les passages les plus proches sémantiquement d'une requête. **ChromaDB** a été retenu pour sa facilité d'intégration en mode HTTP et son support natif des collections isolées par analyse.

2.5 Synthèse et positionnement

GRC AI Analyzer se positionne à l'intersection de trois domaines : la gestion des risques et de la conformité (GRC), le traitement automatique du langage naturel (NLP/LLM) et la recherche sémantique (RAG). Cette combinaison inédite constitue la valeur différenciante de la plateforme par rapport aux solutions existantes.

3. Analyse des besoins et conception

3.1 Introduction

Ce chapitre traduit les exigences opérationnelles en artefacts de conception formels : identification des acteurs et des besoins fonctionnels et non fonctionnels, diagramme des cas d'utilisation, diagramme de classes, diagramme de séquence, diagramme d'activité, architecture globale en couches et diagramme de déploiement. Ces modèles constituent la référence conceptuelle complète de toute la phase d'implémentation.

3.2 Analyse des besoins

3.2.1 Identification des acteurs

La plateforme GRC AI Analyzer implique trois acteurs principaux dont les périmètres d'action sont distincts :

- **L'Analyste GRC** — acteur central du système. Il uploades les rapports (PDF, DOCX, XLSX), déclenche les analyses, consulte les résultats (matrice des risques, taux de conformité, recommandations), interagit avec l'assistant RAG et exporte les livrables.
- **L'Encadrant / Manager** — profil à accès en lecture. Il consulte le tableau de bord, visualise le score de maturité GRC global, suit l'avancement des recommandations et accède aux rapports d'analyse générés par les analystes.
- **L'Administrateur** — responsable de la gouvernance applicative. Il gère les comptes utilisateurs (création, attribution des rôles **ADMIN / ANALYSTE**), consulte la piste d'audit et peut configurer les référentiels de conformité.
- **Mistral AI** — acteur externe (API tierce). Le modèle `mistral-large-latest` est sollicité pour l'extraction des risques, les vérifications de conformité, la génération des recommandations et la production du résumé exécutif.

3.2.2 Besoins fonctionnels

Les besoins fonctionnels ont été organisés en cinq modules cohérents :

Module 1 — Gestion des rapports

- **BF-01** : L'analyste doit pouvoir uploader un fichier au format PDF, DOCX ou XLSX dont la taille ne dépasse pas la limite configurée. Le système valide l'extension et rejette tout format non supporté.
- **BF-02** : Le système enregistre les métadonnées du rapport (nom, format, date d'upload, statut) dans la base de données et stocke le fichier binaire sur le serveur de fichiers local.
- **BF-03** : L'analyste peut consulter l'historique de ses rapports et filtrer par statut (*en_attente*, *en_cours*, *termine*, *erreur*).
- **BF-04** : L'analyste peut supprimer un rapport, ce qui entraîne la suppression en cascade des analyses associées, des fichiers physiques et des collections vectorielles ChromaDB correspondantes.

Module 2 — Analyse IA automatisée

- **BF-05** : À partir d'un rapport uploadé, l'analyste peut lancer une analyse automatique. L'API répond immédiatement (HTTP 202) et l'analyse s'exécute en tâche de fond.
- **BF-06** : Le système extrait le texte brut du document (page par page pour PDF via PyMuPDF, paragraphe par paragraphe pour DOCX via python-docx, feuille par feuille pour XLSX via openpyxl).
- **BF-07** : Le système découpe le texte en fragments (*chunks*) de 1 500 caractères avec un chevauchement de 200 caractères, puis les indexe dans ChromaDB via le modèle d'embedding multilingue *paraphrase-multilingual-MiniLM-L12-v2*.
- **BF-08** : Le système extrait les risques de manière exhaustive : chaque alerte, vulnérabilité, écart ou finding du document constitue un risque distinct, caractérisé par un libellé, une description technique, une catégorie (*CYBER*, *OPERATIONNEL*, *LEGAL*, *FINANCIER*, *RH*), une probabilité (1-5), un impact (1-5) et un score calculé (probabilité $\times$ impact).
- **BF-09** : La sévérité est déterminée automatiquement selon les seuils : score $\geq 20 \rightarrow$ CRITIQUE ; $\geq 12 \rightarrow$ ÉLEVÉ ; $\geq 6 \rightarrow$ MOYEN ; $< 6 \rightarrow$ FAIBLE.
- **BF-10** : Le système vérifie la conformité du document par rapport à trois référentiels en parallèle : ISO 27001 :2013 (14 domaines de l'Annexe A), RGPD (12 domaines), Loi 09-08 marocaine (11 domaines). Chaque domaine reçoit un statut (CONFORME / PARTIEL / NON_CONFORME) et un taux de conformité.
- **BF-11** : Le système génère une recommandation priorisée pour chaque risque identifié, incluant un type d'action (*QUICK_WIN* ou *LONG_TERME*) et un effort estimé (*FAIBLE*, *MOYEN*, *ÉLEVÉ*).

- **BF-12** : Le système produit un résumé exécutif structuré (350-450 mots), destiné à un CISO, synthétisant les conclusions clés, les risques prioritaires, l'état de conformité et les actions immédiates recommandées.
- **BF-13** : Le score de maturité GRC global est calculé comme la moyenne arithmétique des taux de conformité des trois référentiels.

Module 3 — Consultation et visualisation

- **BF-14** : L'analyste peut consulter la matrice des risques (visualisation Probabilité × Impact) via un composant Recharts dédié.
- **BF-15** : L'analyste peut visualiser le taux de conformité par référentiel via un radar chart (composant `ComplianceRadar`).
- **BF-16** : L'analyste peut lire les recommandations classées par priorité (CRITIQUE → HAUTE → MOYENNE → FAIBLE) et mettre à jour leur statut de traitement (*À_FAIRE* → *EN_COURS* → *CLÔTURÉ*).
- **BF-17** : Le tableau de bord affiche un score de maturité global, le nombre de risques par sévérité et les taux de conformité de la dernière analyse.

Module 4 — Assistant RAG conversationnel

- **BF-18** : L'analyste peut poser des questions en langage naturel (français ou anglais) sur le contenu du rapport. Le système récupère les 5 fragments les plus pertinents dans ChromaDB par similarité cosinus, construit un contexte et interroge Mistral pour générer une réponse sourcée.
- **BF-19** : Les réponses du chat citent explicitement les extraits du rapport sur lesquels elles s'appuient.

Module 5 — Export

- **BF-20** : L'analyste peut exporter l'analyse complète en PDF (rapport structuré avec page de couverture, résumé exécutif, tableau des risques, résultats de conformité et recommandations), généré via ReportLab.
- **BF-21** : L'analyste peut exporter le registre des risques en fichier Excel (.xlsx) avec quatre onglets (Résumé, Risques, Conformité, Recommandations), généré via XlsxWriter avec mise en forme conditionnelle par sévérité.

Module 6 — Authentification et administration

- **BF-22** : Le système implémente une authentification par JWT (JSON Web Token). Le token est signé avec l'algorithme HS256, stocké côté client dans le *localStorage* via Zustand, et transmis dans l'en-tête **Authorization: Bearer** à chaque requête.
- **BF-23** : Les mots de passe sont hachés avec bcrypt avant stockage. La vérification à la connexion compare le hash stocké sans jamais manipuler le mot de passe en clair.
- **BF-24** : L'administrateur peut créer et gérer les utilisateurs, leur attribuer le rôle ADMIN ou ANALYSTE. Un analyste ne peut accéder qu'à ses propres rapports ; un administrateur a une vue globale.

3.2.3 Besoins non fonctionnels

Performance et scalabilité

- **BNF-01** : L'API doit répondre aux requêtes de consultation en moins de 500 ms en conditions normales. L'upload et le déclenchement de l'analyse retournent immédiatement (HTTP 202) sans bloquer le client.
- **BNF-02** : Le pipeline d'analyse exploite la concurrence via `asyncio.gather` : l'indexation ChromaDB, l'extraction des risques et les trois vérifications de conformité s'exécutent en parallèle (Phase 1) ; la génération des recommandations et du résumé exécutif s'exécutent elles aussi en parallèle (Phase 2).
- **BNF-03** : Les appels à l'API Mistral sont sérialisés par un sémaphore global (`asyncio.Semaphore(1)`) avec un délai minimum de 1,5 seconde entre appels consécutifs pour respecter les limites de débit (*rate limits*). Un mécanisme de backoff exponentiel gère les erreurs HTTP 429 (délai de $30 \text{ s} \times \text{numéro de tentative}$).

Fiabilité et résilience

- **BNF-04** : L'indexation RAG dans ChromaDB est *non-bloquante* : un échec de ChromaDB n'interrompt pas l'analyse, la plateforme continue sans fonctionnalité de chat.
- **BNF-05** : En cas d'erreur dans le pipeline, le statut de l'analyse est mis à jour en *erreur* dans la base de données et l'erreur est journalisée via Loguru, sans lever d'exception non gérée.
- **BNF-06** : Toutes les sessions de base de données du pipeline asynchrone s'exécutent dans un contexte transactionnel dédié (`AsyncSessionLocal`), distinct de la session HTTP, pour éviter tout conflit de concurrence.

Sécurité

- **BNF-07** : Toutes les routes API (sauf `/auth/login` et `/auth/register`) sont protégées par la dépendance `get_current_user` qui vérifie la validité du JWT.
- **BNF-08** : Les tokens JWT ont une durée de vie de 480 minutes (8 heures). Un intercepteur Axios côté frontend invalide la session locale dès réception d'une réponse HTTP 401.
- **BNF-09** : La politique CORS est configurée explicitement pour n'autoriser que l'origine du frontend (`localhost:5173` en développement, URL configurable en production).

Maintenabilité et extensibilité

- **BNF-10** : L'architecture adopte une séparation stricte des responsabilités : les routes FastAPI sont de simples façades, toute la logique métier réside dans la couche `services/`, et la logique IA dans la couche `ai/`.
- **BNF-11** : Les migrations de base de données sont gérées par Alembic, permettant des évolutions du schéma sans perte de données.
- **BNF-12** : Les prompts Mistral sont externalisés dans des modules dédiés (`risk_prompts.py`, `compliance_prompts.py`, `recommendation_prompts.py`, `summary_prompts.py`), facilitant leur maintenance et leur amélioration indépendante.

Internationalisation

- **BNF-13** : Le modèle d'embedding `paraphrase-multilingual-MiniLM-L12-v2` supporte 50+ langues, permettant l'analyse de documents en français, anglais ou arabe. L'assistant RAG répond dans la langue de la question posée.

3.3 Diagramme des Cas d'Utilisation

3.3.1 Description générale

Le diagramme des cas d'utilisation (figure 3.1) représente l'ensemble des interactions fonctionnelles entre les acteurs et le système GRC AI Analyzer. Il s'organise en six packages fonctionnels qui correspondent aux modules identifiés lors de l'analyse des besoins.

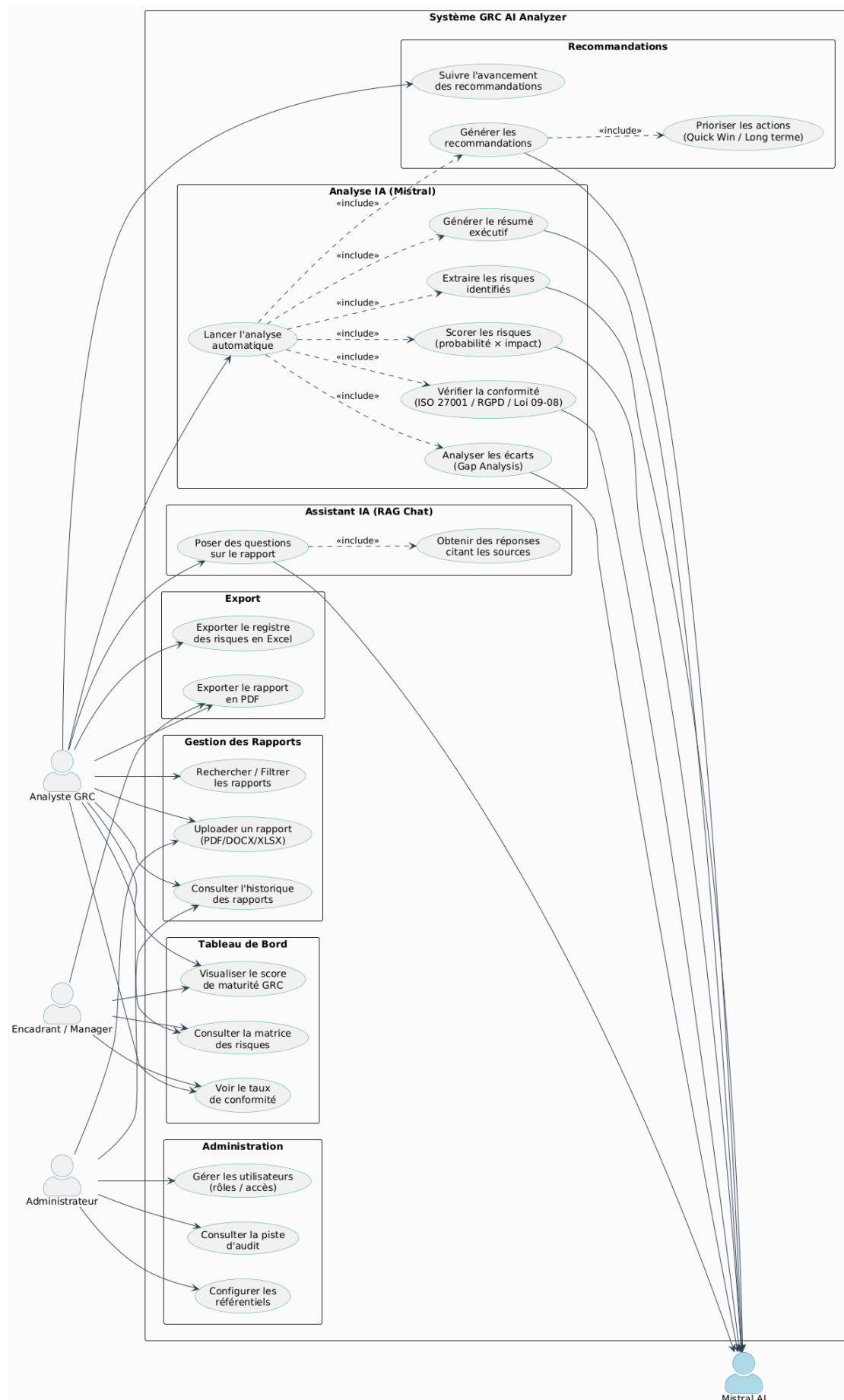


FIGURE 3.1 – Diagramme des cas d'utilisation — GRC AI Analyzer

3.3.2 Description des cas d'utilisation principaux

UC-01 : Uploader un rapport

UC-01 — Uploader un rapport

Acteur principal : Analyste GRC

Précondition : L'analyste est authentifié.

Scénario nominal :

1. L'analyste sélectionne un fichier (PDF, DOCX ou XLSX) via l'interface.
2. Le système valide le format (`validate_file_format`).
3. Le fichier est sauvegardé sur le serveur sous `uploads/reports/{uuid}/{nom}`.
4. Un enregistrement **Rapport** est créé en base avec le statut `en_attente`.
5. L'API retourne l'identifiant du rapport (HTTP 201).

Exception : Format non supporté → HTTP 400 avec message d'erreur.

UC-02 : Lancer l'analyse automatique

UC-02 — Lancer l'analyse automatique

Acteur principal : Analyste GRC

Acteur secondaire : Mistral AI

Précondition : Un rapport au statut `en_attente` existe.

Scénario nominal :

1. L'analyste déclenche l'analyse via `POST /api/v1/analyses/lancer/{rapport_id}`.
2. Le système crée un enregistrement **Analyse** (statut `en_cours`) et retourne immédiatement HTTP 202.
3. En tâche de fond : extraction du texte, chunking, indexation ChromaDB, extraction des risques et vérifications de conformité (Phase 1 parallèle), puis recommandations et résumé (Phase 2 parallèle).
4. L'analyse est marquée *terminé* à la fin du pipeline.

Cas inclus : Extraire les risques identifiés ; Scorer les risques ; Vérifier la conformité ; Analyser les écarts (Gap Analysis) ; Générer le résumé exécutif.

UC-03 : Poser des questions via le Chat IA

UC-03 — Poser des questions sur le rapport (RAG Chat)

Acteur principal : Analyste GRC

Acteur secondaire : Mistral AI

Précondition : L'analyse est terminée et les chunks ont été indexés.

Scénario nominal :

1. L'analyste saisit une question dans l'onglet Assistant RAG.
2. Le système encode la question via le modèle sentence-transformer.
3. ChromaDB retourne les 5 chunks les plus proches par similarité cosinus.
4. Le système construit le prompt contextuel et interroge Mistral.
5. La réponse est affichée avec les extraits sources cités.

Cas inclus : Obtenir des réponses citant les sources.

UC-04 : Exporter les livrables

L'analyste peut exporter deux types de livrables depuis l'interface :

- **Export PDF** — rapport d'analyse complet généré par ReportLab, comprenant une page de couverture avec score de maturité, un résumé exécutif, un tableau des risques avec badges de sévérité colorés, les résultats de conformité par référentiel et le plan de recommandations.
- **Export Excel** — registre des risques au format XLSX (XlsxWriter) avec quatre onglets structurés et mise en forme conditionnelle par sévérité et priorité.

UC-05 : Suivre l'avancement des recommandations

L'analyste peut faire progresser le statut de chaque recommandation via l'interface (bouton de cycle d'état) : *À_FAIRE* → *EN_COURS* → *CLÔTURÉ*, en appelant `PATCH /api/v1/recommandations/{id}/statut`.

3.4 Diagramme de Classes

3.4.1 Vue d'ensemble

Le diagramme de classes (figure 3.2) modélise les entités persistantes du domaine et leurs relations. Il reflète fidèlement le schéma de base de données PostgreSQL géré par

SQLAlchemy (ORM asynchrone) et versionné via Alembic.

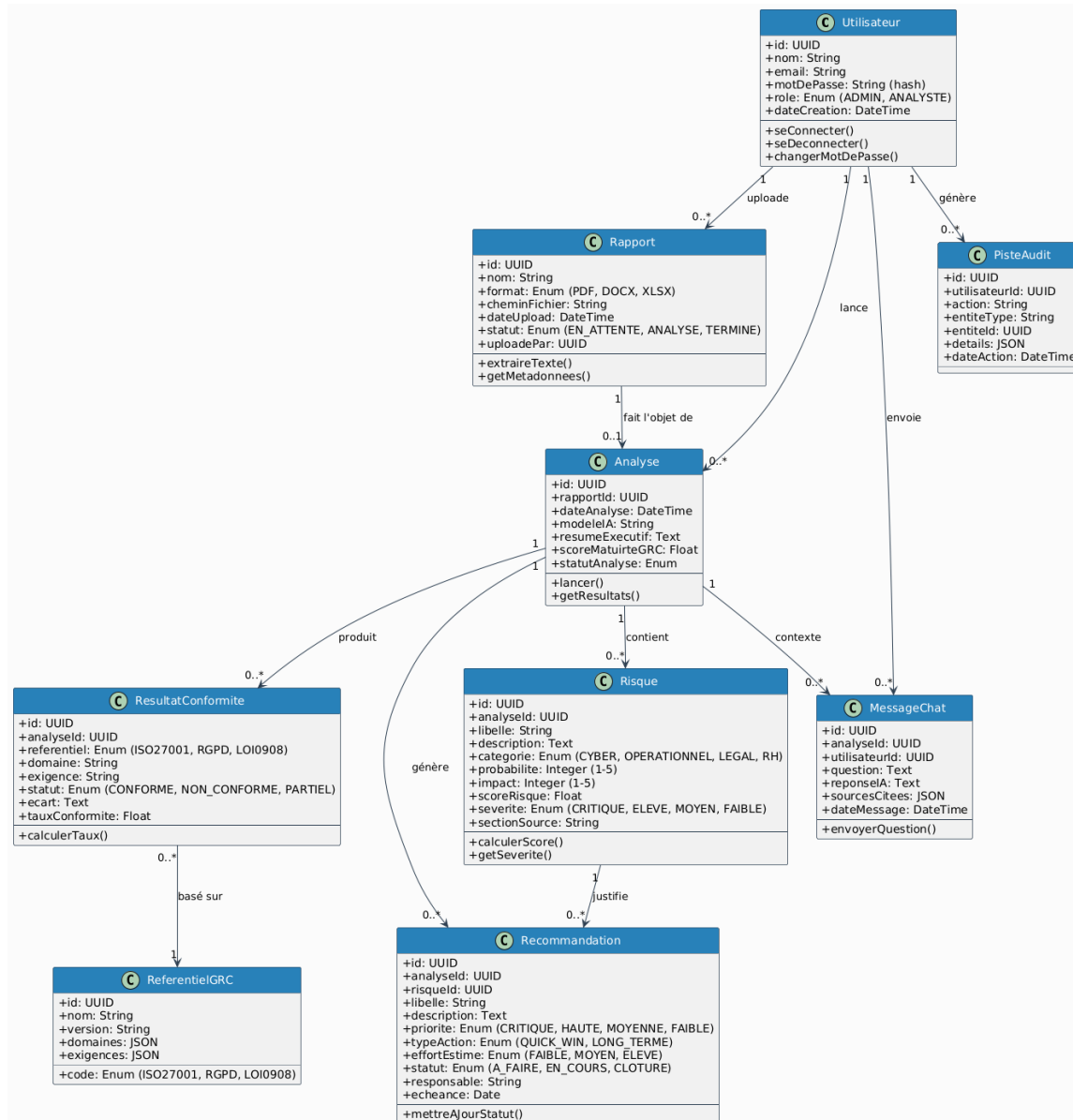


FIGURE 3.2 – Diagramme de classes — modèle du domaine GRC AI Analyzer

3.4.2 Description des classes principales

Utilisateur

La classe *Utilisateur* représente les comptes de la plateforme. Chaque utilisateur possède un identifiant UUID, un nom, un email unique (indexé), un mot de passe haché (bcrypt), un rôle énuméré (*ADMIN* ou *ANALYSTE*) et une date de création. Un utilisateur peut uploader plusieurs rapports et déclencher plusieurs analyses (relations *one-to-many*).

Rapport

La classe **Rapport** encapsule les métadonnées d'un document soumis à analyse. Elle stocke le nom du fichier, le format (**pdf**, **docx**, **xlsx**), le chemin de stockage sur disque (chaîne de type **reports/{uuid}/{nom}**), et le statut du cycle de vie (*en_attente* → *en_cours* → *terminé* ou *erreur*). La suppression d'un rapport entraîne la suppression en cascade de toutes ses analyses associées (**cascade="all, delete-orphan"**).

Analyse

La classe **Analyse** constitue le cœur du modèle. Elle est liée à un rapport (many-to-one) et à un utilisateur déclencheur. Elle porte le résumé exécutif (texte libre), le score de maturité GRC (flottant entre 0 et 100), et le statut de traitement. Elle agrège en composition les risques, les résultats de conformité et les recommandations.

Risque

La classe **Risque** représente une menace, vulnérabilité ou écart identifié par Mistral dans le document. Ses attributs clés sont :

- **libelle** — intitulé du risque (500 caractères max)
- **categorie** — taxonomie en cinq valeurs : CYBER, OPERATIONNEL, LEGAL, FINANCIER, RH
- **probabilite** et **impact** — entiers de 1 à 5
- **score_risque** — produit probabilité × impact (flottant)
- **severite** — calculée automatiquement selon les seuils définis dans **compute_severity()**
- **section_source** — référence à l'identifiant ou à la section du document source (ex. : « Finding #3 », « Section A.12 »)

ResultatConformite

La classe **ResultatConformite** enregistre l'évaluation d'un domaine de conformité pour un référentiel donné. Ses attributs principaux sont le référentiel (ISO27001, RGPD, LOI0908), le nom du domaine évalué, le statut (**CONFORME** / **NON_CONFORME** / **PARTIEL**), l'écart constaté (texte libre) et le taux de conformité (flottant). Une analyse peut produire jusqu'à 37 enregistrements de conformité (14 domaines ISO + 12 RGPD + 11 Loi 09-08).

Recommandation

La classe **Recommandation** modélise une action corrective générée par Mistral en réponse à un risque. Elle est liée optionnellement à un risque (**risque_id** nullable), permettant la traçabilité risque-recommandation. Ses attributs incluent : la priorité (CRITIQUE / HAUTE / MOYENNE / FAIBLE), le type d'action (QUICK_WIN ou LONG_TERME), l'effort estimé (FAIBLE / MOYEN / ÉLEVÉ) et le statut de traitement (À_FAIRE / EN_COURS / CLÔTURÉ).

MessageChat

La classe **MessageChat** (présente dans le modèle conceptuel) stocke l'historique des interactions conversationnelles : la question posée par l'utilisateur, la réponse générée par Mistral et les sources citées (tableau JSON des extraits ChromaDB).

ReferentielGRC

La classe **ReferentielGRC** représente la configuration des cadres normatifs : ISO 27001, RGPD et Loi 09-08. Elle stocke le code du référentiel, sa version, et la liste de ses domaines et exigences au format JSON.

PisteAudit

La classe **PisteAudit** enregistre les actions réalisées par les utilisateurs (upload, lancement d'analyse, modification de statut), permettant à l'administrateur de tracer les opérations sensibles via le type d'entité, l'identifiant de l'entité concernée et les détails en JSON.

3.5 Diagramme de Séquence

3.5.1 Séquence principale : Analyse d'un rapport GRC

Le diagramme de séquence (figure 3.3) décrit les interactions chronologiques entre les composants lors de l'opération la plus complexe du système : le lancement et l'exécution complète d'une analyse IA.

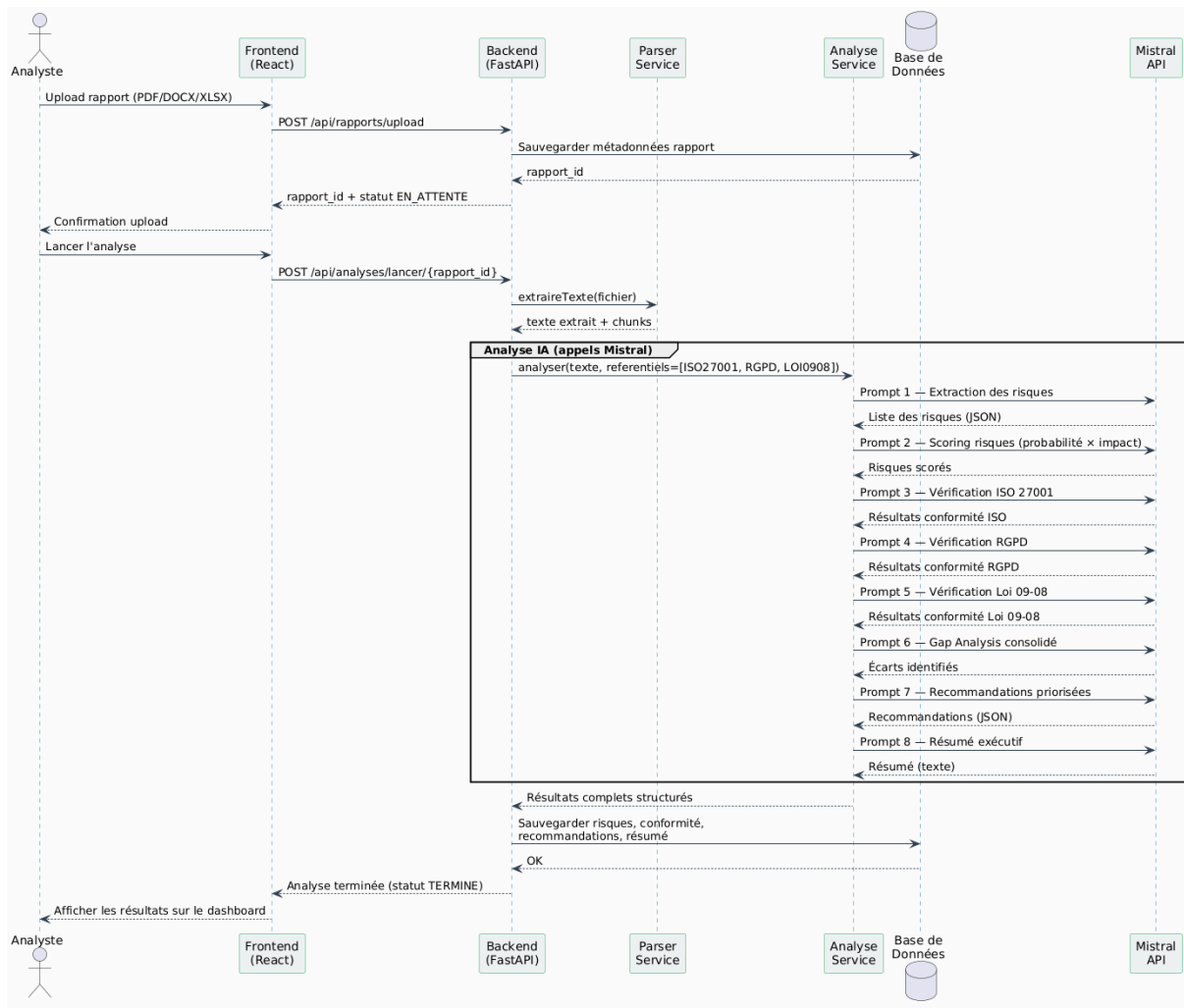


FIGURE 3.3 – Diagramme de séquence — Analyse d'un rapport GRC

3.5.2 Description détaillée des interactions

Le flux se décompose en deux grandes phases, séparées par une frontière asynchrone matérialisée par la réponse HTTP 202.

Phase synchrone : Upload et déclenchement

1. L'analyste soumet son fichier depuis le frontend React via `POST /api/rapports/upload`. Axios joint le token JWT dans l'en-tête de la requête.
2. Le backend FastAPI valide le format du fichier et délègue au `report_service` qui sauvegarde le fichier sur disque et crée l'enregistrement `Rapport` en base de données (statut : `en_attente`).
3. Le frontend reçoit le `rapport_id` et affiche une confirmation d'upload.
4. L'analyste déclenche l'analyse via `POST /api/analyses/lancer/{rapport_id}`.

5. Le backend crée immédiatement l'enregistrement **Analyse** (statut : *en_cours*), enregistre la tâche de fond via **BackgroundTasks**, et retourne HTTP 202 avec l'*analyse_id*.

Phase asynchrone : Pipeline IA (8 appels Mistral)

Le service `analysis_service.run_analysis()` s'exécute dans une session SQLAlchemy dédiée. Il suit le pipeline suivant :

1. **Extraction du texte** — le fichier est lu depuis le disque et parsé selon son format.
Le texte brut est découpé en chunks.
2. **Phase 1 — Parallélisation via `asyncio.gather` :**
 - Indexation ChromaDB (embedding des chunks en mémoire)
 - Prompt 1 → Mistral : extraction exhaustive des risques (JSON structuré)
 - Prompt 2 → Mistral : scoring probabilité × impact des risques
 - Prompt 3 → Mistral : vérification conformité ISO 27001 (14 domaines)
 - Prompt 4 → Mistral : vérification conformité RGPD (12 domaines)
 - Prompt 5 → Mistral : vérification conformité Loi 09-08 (11 domaines)Les résultats sont validés (sévérités, catégories, doublons) et persistés en base de données.
3. **Gap Analysis consolidée** — à partir des taux de conformité calculés, le score de maturité global est déterminé (moyenne des trois référentiels).
4. **Phase 2 — Parallélisation via `asyncio.gather` :**
 - Prompt 7 → Mistral : génération des recommandations priorisées (une par risque identifié)
 - Prompt 8 → Mistral : génération du résumé exécutif (350-450 mots, structuré en 6 sections)
5. **Sauvegarde finale** — l'enregistrement **Analyse** est mis à jour avec le résumé, le score de maturité et le statut *terminé*.

Phase de consultation

Le frontend React interroge `GET /api/analyses/{id}` toutes les 4 secondes (polling) jusqu'à obtenir le statut *terminé*. À ce moment, trois requêtes parallèles (`Promise.all`) récupèrent les risques, les résultats de conformité et les recommandations, qui sont affichés dans les onglets correspondants de la page d'analyse.

3.6 Diagramme d'Activité

3.6.1 Vue d'ensemble du flux de traitement

Le diagramme d'activité (figure 3.4) modélise le flux de contrôle complet du système, de l'upload du rapport jusqu'à la consultation des résultats et l'export des livrables.

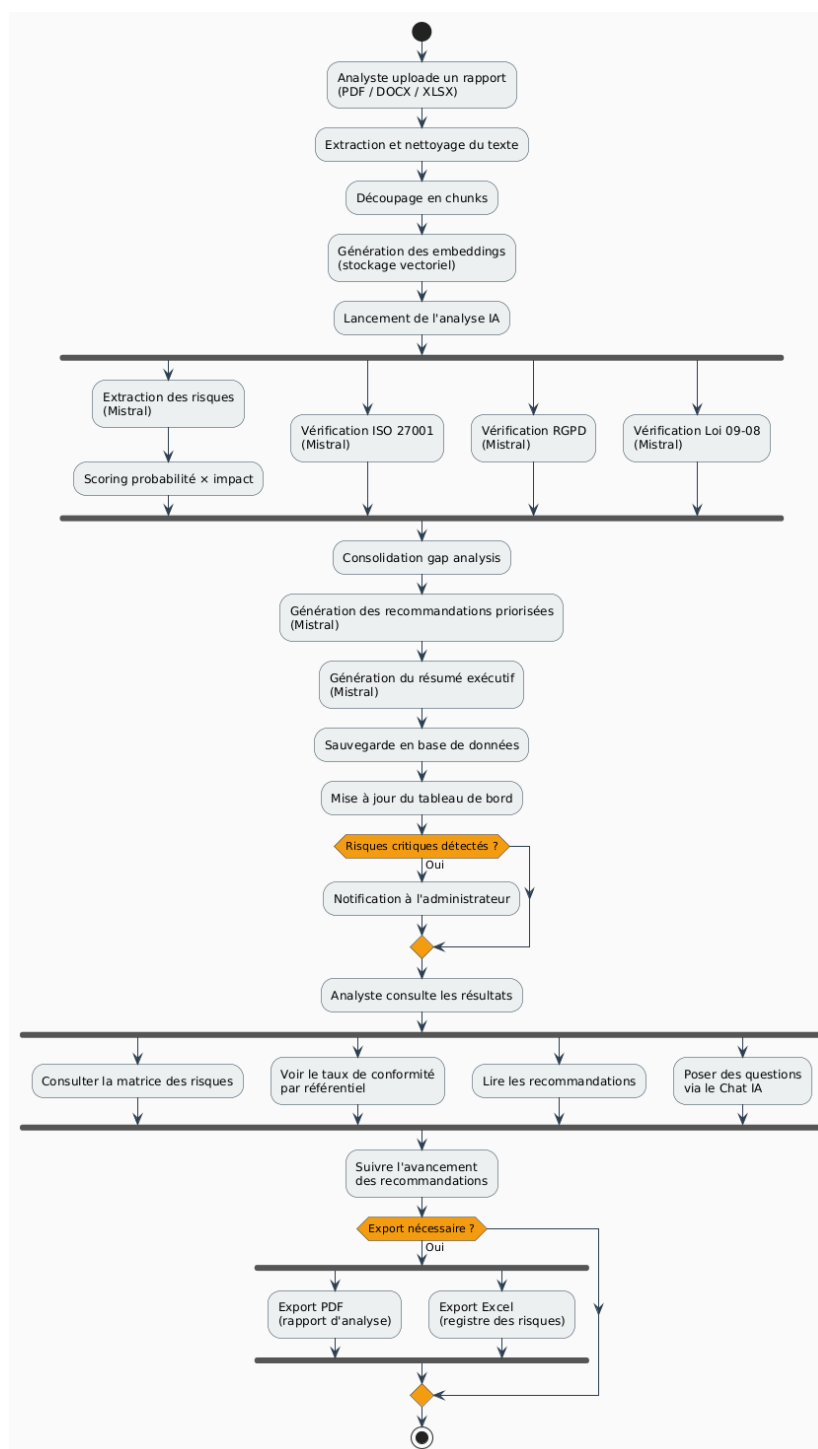


FIGURE 3.4 – Diagramme d'activité — flux de traitement GRC AI Analyzer

3.6.2 Description des activités

Bloc 1 — Préparation du document

Les quatre premières activités forment une séquence linéaire :

1. **Upload du rapport** — l'analyste soumet son fichier PDF/DOCX/XLSX via l'interface. Le système valide le format, sauvegarde le fichier et enregistre les métadonnées.
2. **Extraction et nettoyage du texte** — selon le format détecté, l'un des trois parsers est appelé (PyMuPDF pour PDF, python-docx pour DOCX, openpyxl pour XLSX). Le texte est extrait structuré par page, paragraphe ou cellule.
3. **Découpage en chunks** — le texte est fragmenté en segments de 1 500 caractères avec un chevauchement de 200 caractères assurant la continuité sémantique.
4. **Génération des embeddings** — chaque chunk est transformé en vecteur dense de 384 dimensions par le modèle `paraphrase-multilingual-MiniLM-L12-v2` et indexé dans ChromaDB dans une collection nommée `analyse_{id}`.

Bloc 2 — Analyse IA parallèle (Phase 1)

Un nœud de fourche (*fork*) déclenche simultanément quatre activités parallèles, matérialisant l'utilisation de `asyncio.gather` dans le code :

- **Extraction des risques (Mistral)** — appel JSON structuré retournant la liste des risques avec libellé, description, catégorie, probabilité, impact et référence source.
- **Scoring probabilité × impact** — calcul déterministe du score ($score = probabilité \times impact$) et classification automatique de la sévérité selon les seuils définis.
- **Vérification ISO 27001 (Mistral)** — évaluation des 14 domaines de l'Annexe A avec statut CONFORME/PARTIEL/NON_CONFORME et taux par domaine.
- **Vérification RGPD (Mistral)** — évaluation des 12 articles RGPD pertinents.
- **Vérification Loi 09-08 (Mistral)** — évaluation des 11 domaines de la loi marocaine de protection des données.

Un nœud de jonction (*join*) synchronise les cinq activités parallèles avant de passer à la consolidation.

Bloc 3 — Consolidation et génération (Phase 2)

1. **Consolidation gap analysis** — les taux de conformité sont agrégés, le score de maturité global est calculé.

2. **Génération des recommandations priorisées (Mistral) et génération du résumé exécutif (Mistral)** s'exécutent en parallèle.
3. **Sauvegarde en base de données** — tous les objets (risques, conformités, recommandations, résumé) sont persistés dans PostgreSQL.
4. **Mise à jour du tableau de bord** — le statut de l'analyse passe à *terminé*.

Décision : Risques critiques détectés ?

Après la mise à jour du tableau de bord, un nœud de décision vérifie la présence de risques de sévérité CRITIQUE (score ≥ 20). Si oui, une notification est envoyée à l'administrateur. Dans les deux cas, l'analyste est redirigé vers la page des résultats.

Bloc 4 — Consultation des résultats (parallèle)

L'analyste dispose de quatre vues accessibles simultanément via les onglets de l'interface :

- **Matrice des risques** — visualisation Probabilité $\times$ Impact via RiskMatrix (Recharts).
- **Taux de conformité par référentiel** — radar chart via ComplianceRadar (Recharts).
- **Recommandations** — plan d'action priorisé avec gestion du cycle de vie.
- **Chat IA** — assistant conversationnel RAG (Mistral + ChromaDB).

Puis l'analyste peut suivre l'avancement des recommandations au fil du temps.

Décision : Export nécessaire ?

Si l'analyste souhaite exporter, un nœud de fourche permet de générer en parallèle le rapport PDF (ReportLab) et/ou le registre Excel (XlsxWriter), avant de converger vers la fin du flux.

3.7 Architecture globale de la solution

3.7.1 Vue d'ensemble en couches

La figure 3.5 présente l'architecture globale de GRC AI Analyzer, organisée autour de quatre grandes zones fonctionnelles qui collaborent pour transformer un document GRC brut en analyse structurée, exploitable et conversationnelle.

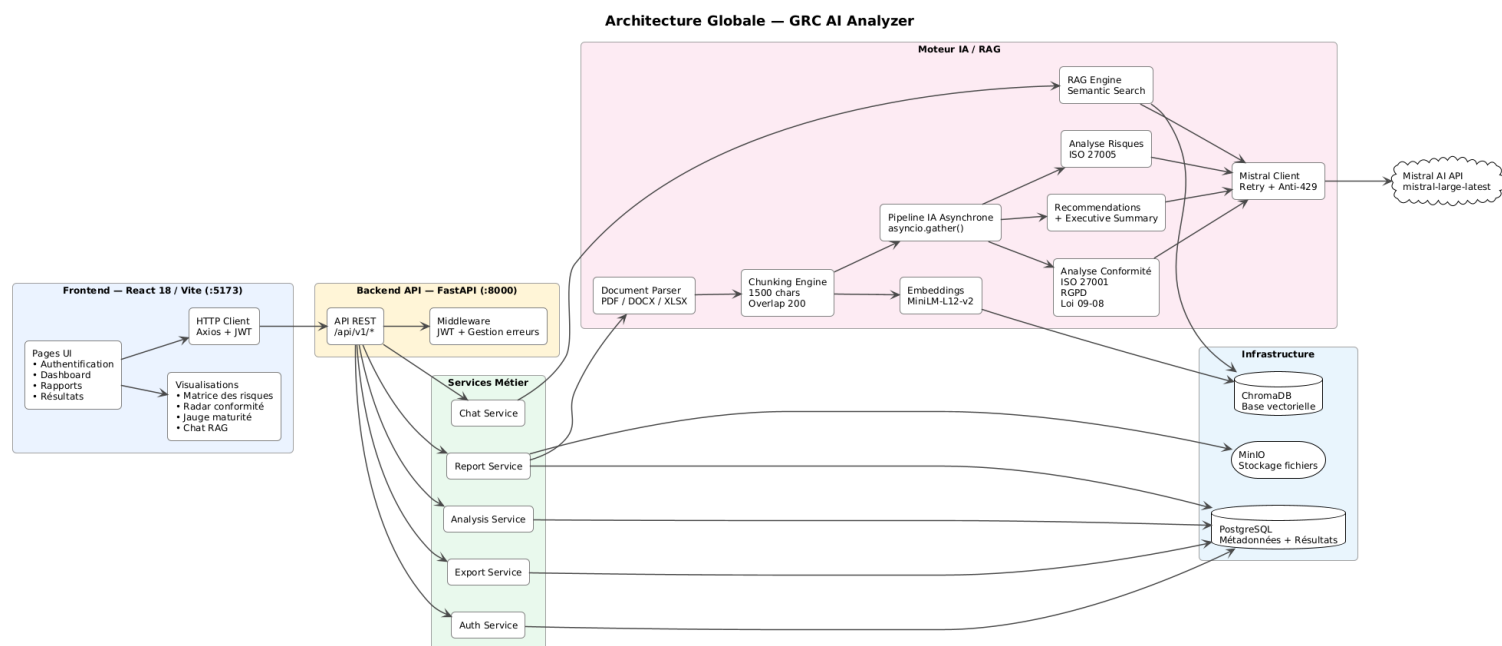


FIGURE 3.5 – Architecture globale — GRC AI Analyzer

Zone Frontend — React 18 / Vite (:5173). Le frontend est une Single Page Application React exposant quatre catégories de vues : authentification, tableau de bord, gestion des rapports et consultation des résultats. Un **HTTP Client Axios + JWT** injecte automatiquement le token d'accès dans chaque requête sortante. Les composants de visualisation (matrice des risques, radar de conformité, jauge de maturité, chat RAG) consomment les données retournées par l'API sans logique métier locale.

Zone Backend API — FastAPI (:8000). Le backend expose un point d'entrée REST unifié `/api/v1/*` protégé par un **Middleware JWT + Gestion des erreurs**. Cinq services métier distincts sont orchestrés depuis cette couche : *Chat Service* (assistant RAG), *Report Service* (stockage et cycle de vie des rapports), *Analysis Service* (déclenchement et suivi du pipeline IA), *Export Service* (génération PDF et Excel), et *Auth Service* (inscription, connexion, gestion des rôles RBAC).

Zone Moteur IA / RAG. C'est le cœur différenciant de la plateforme. Le flux commence par le **Document Parser** (PDF via PyMuPDF, DOCX via python-docx, XLSX via openpyxl), puis le **Chunking Engine** (1 500 caractères, chevauchement de 200) et la génération d'**Embeddings** par le modèle multilingue MiniLM-L12-v2. Le **Pipeline IA Asynchrone** (`asyncio.gather()`) lance ensuite en parallèle : l'indexation RAG (*RAG Engine / Semantic Search*), l'extraction des risques selon ISO 27005, l'évaluation de la conformité (ISO 27001, RGPD, Loi 09-08), et la génération des recommandations et du résumé exécutif. Tous ces appels transitent par le **Mistral Client** doté d'un mécanisme

de retry et d'un anti-429 (sémaphore + backoff exponentiel), avant d'être acheminés vers l'API Mistral AI (mistral-large-latest).

Zone Infrastructure. Trois moteurs de persistance spécialisés : **ChromaDB** (base vectorielle, index cosinus HNSW, une collection par analyse), **MinIO** (stockage objet compatible S3 pour les fichiers originaux), et **PostgreSQL** (métadonnées relationnelles et résultats d'analyse via SQLAlchemy 2.0 asyncpg). Chaque service métier accède directement au moteur de persistance qui correspond à la nature de ses données.

3.7.2 Diagramme de déploiement

Le diagramme de déploiement (figure 3.6) représente la distribution physique des composants logiciels sur les nœuds d'exécution et les protocoles de communication entre eux.

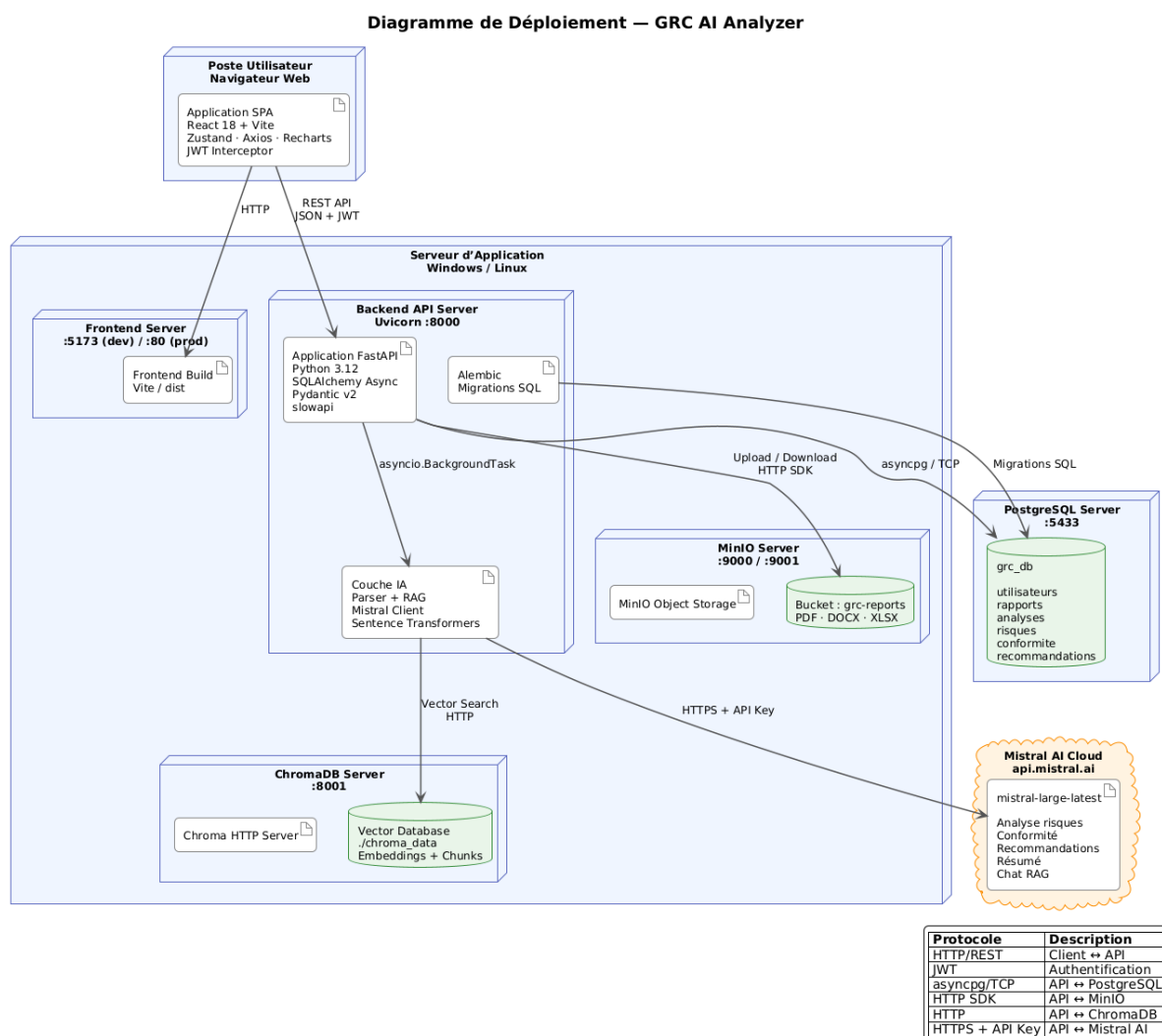


FIGURE 3.6 – Diagramme de déploiement — GRC AI Analyzer

Cinq nœuds structurent le déploiement de la plateforme :

Poste utilisateur — Navigateur Web. Le client exécute la Single Page Application React 18 + Vite. Les bibliothèques Zustand, Axios et Recharts sont chargées dans le navigateur. L'intercepteur JWT injecte automatiquement le token dans chaque requête HTTP. La communication avec le backend s'effectue via **HTTP/REST + JSON + JWT**.

Serveur d'Application (Windows / Linux). Ce nœud héberge deux composants distincts :

- **Frontend Server (:5173 dev / :80 prod)** : sert le build statique Vite (`dist/`) en production.
- **Backend API Server — Uvicorn (:8000)** : exécute l'application FastAPI (Python 3.12, SQLAlchemy Async, Pydantic v2, slowapi). Les analyses longues sont déléguées à `asyncio.BackgroundTask` pour ne pas bloquer la boucle d'événements. Le composant **Alembic Migrations SQL** est invoqué au démarrage pour maintenir le schéma PostgreSQL à jour. La **Couche IA** embarquée regroupe le parser de documents, le moteur RAG, le client Mistral et les Sentence Transformers (modèle `MiniLM-L12-v2`).

MinIO Server (:9000 / :9001). Moteur de stockage objet compatible S3. Le bucket `grc-reports` reçoit les fichiers originaux (PDF, DOCX, XLSX) via le **HTTP SDK MinIO**. Le port 9001 expose la console d'administration.

PostgreSQL Server (:5433). Base de données relationnelle hébergeant la base `grc_db` avec six tables principales : `utilisateurs`, `rapports`, `analyses`, `risques`, `conformite` et `recommandations`. Les accès runtime utilisent le driver `asyncpg/TCP` ; les migrations Alembic utilisent `psycopg2` (synchrone).

ChromaDB Server (:8001). Base vectorielle exécutée en mode HTTP (`chroma run`). Le serveur Chroma HTTP reçoit les requêtes d'indexation (chunks + embeddings) et de recherche sémantique (top-*k* cosinus) depuis la couche IA via **HTTP**. Les vecteurs et chunks sont persistés dans `./chroma_data`.

Mistral AI Cloud (`api.mistral.ai`) — acteur externe. Seul composant extérieur au périmètre de déploiement local. Le modèle `mistral-large-latest` est invoqué via

HTTPS + API Key pour cinq tâches : analyse des risques, évaluation de conformité, recommandations, résumé exécutif et chat RAG. Aucune donnée sensible n'est stockée côté Mistral ; seuls des extraits de texte anonymisés transitent dans les prompts.

Protocoles de communication. Le tableau de légende du diagramme synthétise les protocoles utilisés : HTTP/REST + JWT (client ↔ API), asyncpg/TCP (API ↔ PostgreSQL), HTTP SDK (API ↔ MinIO), HTTP (API ↔ ChromaDB), et HTTPS + API Key (API ↔ Mistral AI).

3.8 Synthèse du chapitre

Ce chapitre a établi les fondements conceptuels de GRC AI Analyzer. Les 24 besoins fonctionnels et 13 besoins non fonctionnels ont été identifiés et organisés en modules cohérents. Les diagrammes UML — cas d'utilisation, classes, séquence, activité, architecture globale et déploiement — constituent la référence complète de toute la phase d'implémentation. Le chapitre suivant détaille les choix techniques et l'architecture retenue.

4. Conception technique et choix technologiques

Ce chapitre suit le fil de la donnée : depuis l'upload d'un document jusqu'à la réponse du chat RAG. Pour chaque étape, les choix technologiques sont justifiés, les décisions de conception exposées, et les extraits de code représentatifs commentés.

4.1 Pile technologique retenue

GRC AI Analyzer repose sur cinq couches technologiques sélectionnées pour satisfaire trois contraintes : **performance** (pipeline IA parallèle), **robustesse** (persistance multi-moteurs) et **sécurité** (JWT, bcrypt, validation stricte, rate limiting).

Couche	Technologie	Rôle / justification
Interface	React 18 + Vite + TypeScript	SPA avec HMR, typage statique, code-splitting
Interface	Zustand + Axios	État global JWT, intercepteur token auto-injecté
Interface	Recharts	Matrice risques (ScatterChart) et radar conformité
Backend	FastAPI 0.115 + Python 3.12	ASGI async, doc OpenAPI auto, BackgroundTask
Backend	SQLAlchemy 2 + asyncpg	ORM async natif, zéro thread bloquant
Backend	Alembic + psycopg2	Migrations versionnées (driver sync pour Alembic)
Backend	Pydantic v2	Validation entrées, enveloppe {data,message,success}
IA / NLP	Mistral Large (API cloud)	LLM central : extraction, conformité, chat RAG
IA / NLP	sentence-transformers MiniLM	Embeddings 384 dims, 50+ langues, singleton

Couche	Technologie	Rôle / justification
Infrastructure	PostgreSQL 15	SGBDR principal, 6 entités UUID, enums natifs
Infrastructure	MinIO (S3-compatible)	Stockage objet des fichiers bruts
Infrastructure	ChromaDB (HTTP :8001)	Base vectorielle HNSW cosinus, 1 collection/analyse
Sécurité	JWT HS256 + bcrypt	Tokens stateless, salt aléatoire par appel
Sécurité	slowapi + magic bytes	Rate limiting IP, validation signature binaire

TABLE 4.1 – Pile technologique de GRC AI Analyzer

4.2 Architecture en couches du backend

Le backend est organisé en cinq couches verticales strictement séparées : une couche ne peut appeler que les couches sous-jacentes, ce qui garantit la testabilité unitaire de chaque niveau.

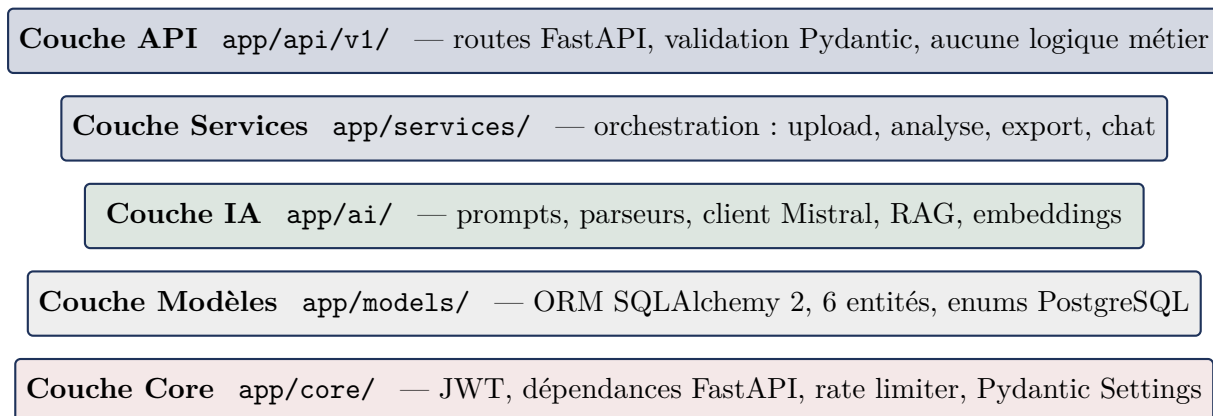


FIGURE 4.1 – Organisation en cinq couches du backend FastAPI

Le point d'entrée `main.py` enregistre les cinq routeurs et configure les middlewares globaux :

```

1  app = FastAPI(title="GRC AI Analyzer", version="1.0.0")
2
3  # Rate limiter global -- HTTP 429 si dépassement par IP
4  app.state.limiter = limiter
5  app.add_exception_handler(RateLimitExceeded,
6                             _rate_limit_exceeded_handler)
7
8  # CORS restreint aux origines connues (frontend Vite :5173 et prod)

```

```

8  app.add_middleware(CORSMiddleware,
9  allow_origins=[settings.frontend_url, "http://localhost:5173"],
10 allow_credentials=True, allow_methods=["*"], allow_headers=["*"])
11
12 # Cinq routeurs métier, tous sous le préfixe /api/v1
13 app.include_router(auth.router, prefix="/api/v1") # /login, /
    register
14 app.include_router(rapports.router, prefix="/api/v1") # /rapports/*
15 app.include_router(analyses.router, prefix="/api/v1") # /analyses/*
16 app.include_router(chat.router, prefix="/api/v1") # /chat/*
17 app.include_router(export.router, prefix="/api/v1") # /export/*

```

Listing 4.1 – Configuration CORS, rate-limit et routeurs (main.py)

La configuration globale est centralisée via Pydantic Settings, ce qui garantit que le fichier `.env` est résolu indépendamment du répertoire courant au démarrage :

```

1  class Settings(BaseSettings):
2  database_url: str # postgresql+asyncpg://...
3  mistral_api_key: str
4  mistral_model: str = "mistral-large-latest"
5  secret_key: str # secrets.token_hex(32)
6  algorithm: str = "HS256"
7  access_token_expire_minutes: int = 480
8  minio_endpoint: str = "localhost:9000"
9  minio_bucket: str = "grc-reports"
10 frontend_url: str = "http://localhost:5173"
11 chroma_host: str = "localhost"
12 chroma_port: int = 8001
13
14 model_config = SettingsConfigDict(
15 # Chemin absolu : fonctionne quel que soit le cwd
16 env_file=Path(__file__).parent.parent / ".env",
17 extra="ignore"
18 )
19
20 settings = Settings() # singleton importe dans toute l'application

```

Listing 4.2 – Configuration via Pydantic Settings (app/config.py)

4.3 Conception du modèle de données

Le modèle relationnel repose sur six entités PostgreSQL reliées par clés étrangères UUID v4.

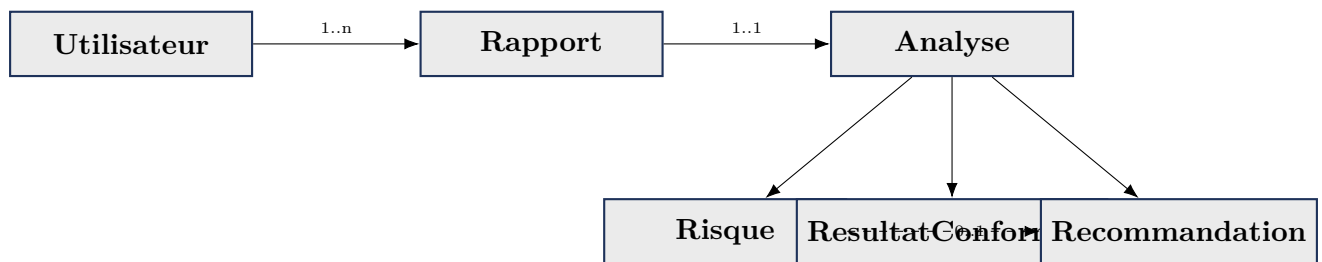


FIGURE 4.2 – Modèle relationnel de GRC AI Analyzer (PostgreSQL)

L'entité **Analyse** est le pivot : elle agrège les résultats des trois axes GRC. La relation optionnelle **Risque** → **Recommandation** (**risque_id** FK nullable) permet à Mistral de lier une recommandation à un risque précis tout en autorisant des recommandations globales non liées.

```

1  class Risque(Base):
2      __tablename__ = "risques"
3
4      id:          Mapped[uuid.UUID] = mapped_column(
5          UUID(as_uuid=True), primary_key=True,
6          default=uuid.uuid4)
7      analyse_id:  Mapped[uuid.UUID] = mapped_column(
8          ForeignKey("analyses.id", ondelete="CASCADE"))
9      libelle:      Mapped[str]      = mapped_column(String(500))
10     categorie:     Mapped[str]      = mapped_column(
11         Enum("CYBER", "OPERATIONNEL", "LEGAL", "FINANCIER", "RH",
12             name="categorie_risque_enum"))
13     probabilite:   Mapped[int]      = mapped_column(Integer)      # 1--5
14     impact:        Mapped[int]      = mapped_column(Integer)      # 1--5
15     score_risque:  Mapped[float]    = mapped_column(Float)        # prob x impact
16     severite:      Mapped[str]      = mapped_column(
17         Enum("CRITIQUE", "ELEVÉ", "MOYEN", "FAIBLE",
18             name="severite_enum"))
19     section_source: Mapped[Optional[str]] = mapped_column(String(500))

```

Listing 4.3 – Modèle ORM Risque — SQLAlchemy 2 (models/risque.py)

La clause `ondelete="CASCADE"` garantit la suppression automatique en cascade : supprimer une analyse entraîne la suppression de tous ses risques, résultats et recommandations sans logique applicative supplémentaire. La formule de sévérité est déterministe :

$$\text{score} = p \times i, \quad p, i \in \{1, \dots, 5\} \quad \text{sévérité} = \begin{cases} \text{FAIBLE} & \text{score} < 6 \\ \text{MOYEN} & 6 \leq \text{score} < 12 \\ \text{ÉLEVÉ} & 12 \leq \text{score} < 20 \\ \text{CRITIQUE} & \text{score} \geq 20 \end{cases}$$

4.4 Conception des routes API

Toutes les routes suivent la convention `/api/v1/{ressource}` et retournent une enveloppe JSON uniforme `{"data": ..., "message": "...", "success": bool}`. Le tableau 4.3 récapitule les endpoints principaux.

Méthode	Route	Description
POST	<code>/auth/login</code>	Authentification ; retourne un token JWT HS256
POST	<code>/auth/register</code>	Création de compte utilisateur
POST	<code>/rapports/upload</code>	Upload PDF/DOCX/XLSX vers MinIO
GET	<code>/rapports/</code>	Liste des rapports de l'utilisateur
DELETE	<code>/rapports/{id}</code>	Supprime rapport + analyse + collection ChromaDB
POST	<code>/analyses/lancer/{rapport_id}</code>	Déclenche l'analyse en arrière-plan (HTTP 202)
GET	<code>/analyses/{id}</code>	Statut et résumé exécutif (<code>en_cours</code> / <code>termine</code>)
GET	<code>/analyses/{id}/risks</code>	Liste des risques extraits
GET	<code>/analyses/{id}/conformite</code>	Résultats de conformité par référentiel
GET	<code>/analyses/{id}/recommandations</code>	Recommandations priorisées
POST	<code>/chat/{analyse_id}</code>	Question RAG sur le rapport analysé
GET	<code>/export/{id}/pdf</code>	Export PDF du rapport complet
GET	<code>/export/{id}/excel</code>	Export Excel (audit trail)

TABLE 4.2 – Routes API principales de GRC AI Analyzer

4.4.1 Lancement d'une analyse (HTTP 202 + BackgroundTask)

La route la plus importante est POST /analyses/lancer/{rapport_id} : elle retourne immédiatement un HTTP 202 pendant que l'analyse tourne en arrière-plan dans une session de base de données indépendante.

```
1  @router.post("/lancer/{rapport_id}", status_code=202)
2  async def lancer_analyse(
3      rapport_id: uuid.UUID,
4      background_tasks: BackgroundTasks,
5      db: Annotated[AsyncSession, Depends(get_db)],
6      current_user: Annotated[Utilisateur, Depends(get_current_user)],
7  ):
8      # Verification existence du rapport
9      result = await db.execute(
10         select(Rapport).where(Rapport.id == rapport_id)
11     )
12     rapport = result.scalar_one_or_none()
13     if not rapport:
14         raise HTTPException(status_code=404, detail="Rapport not found")
15
16     # Creation de l'enregistrement analyse avec statut en_cours
17     analyse = Analyse(
18         id=uuid.uuid4(),
19         rapport_id=rapport_id,
20         lance_par=current_user.id,
21         statut="en_cours",
22     )
23     db.add(analyse)
24     rapport.statut = "en_cours"
25     await db.commit() # commit avant le background task
26
27     # Lancement en arriere-plan (nouvelle session DB independante)
28     background_tasks.add_task(
29         _run_analysis_background, analyse.id, rapport_id
30     )
31     # Reponse immediate : le frontend va poller GET /analyses/{id}
32     return api_response(
33         data={"analyse_id": str(analyse.id), "statut": "en_cours"},
34         message="Analysis started",
35     )
```

Listing 4.4 – Route de lancement d'analyse avec BackgroundTask (analyses.py)

Le commit *avant* l'ajout de la tâche de fond est essentiel : le background task ouvre sa propre session `AsyncSessionLocal` et doit trouver l'enregistrement en base. Sans ce commit, la tâche de fond échouerait avec une erreur de clé étrangère.

4.4.2 Polling du statut d'analyse (frontend)

Le frontend React poll GET /analyses/{id} toutes les 3 secondes jusqu'à statut `=== "termine"` :

```

1  @router.get("/{analyse_id}")
2  async def get_one(
3  analyse_id: uuid.UUID,
4  db: Annotated[AsyncSession, Depends(get_db)],
5  current_user: Annotated[Utilisateur, Depends(get_current_user)],
6  ):
7  analyse = await get_analyse(db, analyse_id)
8  if not analyse:
9  raise HTTPException(status_code=404, detail="Analyse not found")
10 return api_response(data={
11     "id": str(analyse.id),
12     "rapport_id": str(analyse.rapport_id),
13     "resume_executif": analyse.resume_executif,
14     "score_maturite": analyse.score_maturite,
15     "statut": analyse.statut, # "en_cours"/"termine"/"erreur"
16     "created_at": analyse.created_at.isoformat(),
17 })

```

Listing 4.5 – Endpoint de polling du statut d'analyse (analyses.py)

4.4.3 Interface de chat RAG

L'intercepteur Axios injecte automatiquement le token JWT dans toutes les requêtes sortantes :

```

1  const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });
2
3  // Injection automatique du token a chaque requete
4  api.interceptors.request.use((config) => {
5      const token = useAuthStore.getState().token;
6      if (token) config.headers.Authorization = `Bearer ${token}`;
7      return config;
8  });
9
10 // Deconnexion automatique si le token expire (401)
11 api.interceptors.response.use(
12 (res) => res,
13 (err) => {
14     if (err.response?.status === 401) {
15         useAuthStore.getState().logout();
16         window.location.href = "/login";

```

```

17   }
18   return Promise.reject(err);
19 }
20 );

```

Listing 4.6 – Intercepteur JWT Axios (frontend/src/services/api.ts)

Ce mécanisme centralise la gestion du cycle de vie de l'authentification : aucun composant React n'a à gérer explicitement le token.

4.5 Conception des prompts IA

GRC AI Analyzer utilise quatre prompts distincts envoyés à Mistral Large. Chaque prompt est composé d'un *system prompt* (rôle et règles strictes) et d'un *user prompt* (données et format de sortie attendu).

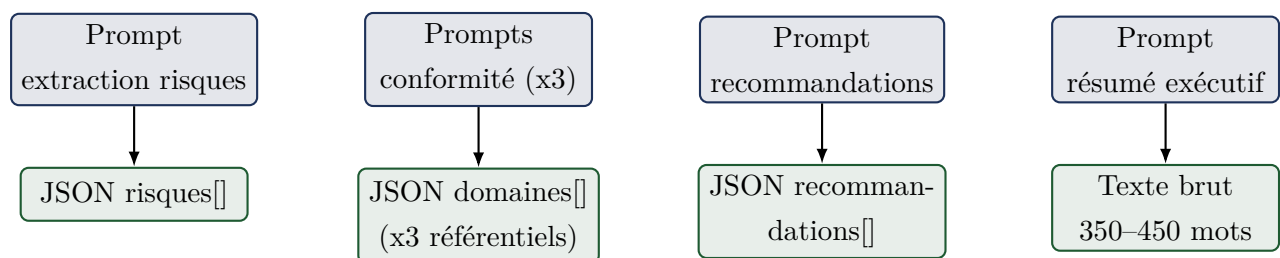


FIGURE 4.3 – Les quatre prompts Mistral et leurs sorties

4.5.1 Prompt d'extraction des risques

Le prompt d'extraction est le plus complexe : il détecte automatiquement le type de document (scan de vulnérabilités, rapport de pentest, audit, registre des risques, incident...) et adapte sa stratégie d'extraction en conséquence.

```

1  RISK_SYSTEM_PROMPT = ""You are a senior GRC and cybersecurity expert.
2  Your task is to perform an EXHAUSTIVE extraction of every risk,
3  vulnerability, finding, gap, weakness, and issue in the document.
4
5  STEP 1 -- IDENTIFY THE DOCUMENT TYPE and apply the matching strategy:
6
7  TYPE A -- VULNERABILITY SCAN (ZAP, Nessus, Burp, Qualys)
8  - Extract EVERY individual alert -- one risk per alert.
9  - DO NOT merge similar alerts (four CSP issues = four risks).
10 - Severity mapping: Critical->P5/I5, High->P4/I4, Medium->P3/I3,
11 Low->P2/I2, Informational->P1/I2.
12

```

```

13  TYPE B -- PENETRATION TEST REPORT
14  - Extract each finding with CVSS score and proof-of-concept.
15  - CVSS: Critical(9+)->P5/I5, High(7-8.9)->P4/I4, Medium->P3/I3.
16
17  TYPE C -- AUDIT / GAP ANALYSIS
18  - Major non-conformance->P4/I4, Minor->P3/I3, Observation->P2/I2.
19  - Specify the exact control/clause not met.
20
21  TYPE D -- RISK REGISTER
22  - Preserve original probability/impact scores when provided.
23
24  TYPE G -- DPIA / PRIVACY REPORT
25  - Reference specific GDPR/Loi 09-08 article for each risk.
26
27  UNIVERSAL RULES:
28  1. EXHAUSTIVE : nothing is too minor to extract.
29  2. NO MERGING : each distinct finding = one risk object.
30  3. GROUNDED   : every risk must be supported by document text.
31  4. Respond ONLY in valid JSON. No text outside the JSON.

```

Listing 4.7 – System prompt d'extraction des risques (risk_prompts.py)

```

1  RISK_USER_PROMPT = """Perform an exhaustive risk extraction.
2
3  Return ONLY this JSON (no text outside):
4  {
5      "document_type": "VULNERABILITY_SCAN|PENTEST|AUDIT|RISK_REGISTER|...
6      ",
7      "risques": [
8          {
9              "libelle":          "Exact name of the finding (max 200 chars)",
10             "description":      "Full technical detail: CWE/CVE IDs, affected
11             URLs,
12             instance count, scanner evidence, remediation.
13             Minimum 100 chars, maximum 1000 chars.",
14             "categorie":        "CYBER|OPERATIONNEL|LEGAL|FINANCIER|RH",
15             "probabilite":       3,
16             "impact":            3,
17             "section_source":    "Alert ID, section, page, or finding ID",
18             "confiance":         90
19         }
20     ]
21 }
22
23 SCORING:
24 - probabilite : likelihood of exploitation (1=rare, 5=near-certain)
25 - impact      : business/security impact if it occurs (1=negligible,
26                 5=catastrophic)

```

```

24 - confiance : how clearly stated in document (0-100; use >=60 for
    explicit)
25
26 DOCUMENT TEXT:
27 {text}"""

```

Listing 4.8 – User prompt d'extraction des risques (risk_prompts.py)

Le champ **confiance** (0–100) est un méta-indicateur : il permet au service métier de filtrer les risques peu fiables (confiance < 40) avant de les persister, évitant d'enregistrer des extrapolations hasardeuses de Mistral.

4.5.2 Prompts de vérification de conformité

Le même template est utilisé trois fois, paramétré par le référentiel (ISO27001, RGPD, LOI0908). Le system prompt instaure une philosophie d'évaluation en deux formes d'évidence :

```

1 COMPLIANCE_SYSTEM_PROMPT = """You are a {framework} compliance expert.
2
3 EVALUATION PHILOSOPHY -- evaluate BOTH forms of evidence:
4
5 1. EXPLICIT evidence: policies mentioned, controls described,
6 the document directly addresses the requirement.
7
8 2. IMPLICIT evidence: findings IMPLY the compliance state even
9 without explicit mention. Examples:
10 - Vulnerability scan showing missing HSTS -> A.13 weak
11 - Audit finding CSRF vulnerabilities -> access control gaps
12 - Incident report describing a breach -> A.16 incident mgmt risk
13
14 SCORING GUIDE:
15 - CONFORME (75-100%) : clear evidence of implementation.
16 - PARTIEL (10-74%) : partial or implicit evidence exists.
17 - NON_CONFORME (0-9%) : no evidence, or explicit failure.
18
19 taux_global = weighted average of all domain scores.
20 Respond ONLY in valid JSON. No text outside the JSON."""

```

Listing 4.9 – System prompt de conformité — philosophie d'évaluation (compliance_prompts.py)

```

1 COMPLIANCE_USER_PROMPT = """Evaluate compliance with {
    framework_display}.
2
3 {domains}

```

```

4  # Exemple pour ISO 27001 :
5  # A.5  Information Security Policies
6  # A.9  Access Control
7  # A.12 Operational Security (vulnerability mgmt, logging, patching)
8  # A.16 Information Security Incident Management
9  # ... (14 domaines au total)
10
11 Return ONLY this JSON:
12 {
13   "referentiel": "{framework}",
14   "taux_global": 0,
15   "domaines": [
16     {
17       "nom":      "Domain name",
18       "statut":   "CONFORME|NON_CONFORME|PARTIEL",
19       "taux":     0,
20       "ecart":    "Specific gap description referencing document content
21                   .",
22       "evidence": "Direct quote or reference from the document."
23     }
24   ]
25
26   IMPORTANT: list ALL domains. taux_global = rounded average of taux[].
27
28   DOCUMENT TEXT:
29   {text}"""

```

Listing 4.10 – User prompt de conformité ISO 27001 (extrait) (compliance_prompts.py)

La distinction `ecart` / `evidence` est délibérée : `ecart` décrit le problème (ce qui manque), `evidence` cite le document (ce qui justifie le score). Cette séparation permet au frontend d'afficher une explication actionnable et une référence source distinctes dans le tableau de conformité.

4.5.3 Prompt de génération des recommandations

Le prompt de recommandations reçoit la liste des risques extraits en phase 1 et exige une recommandation *technique et spécifique* par risque :

```

1  RECOMMENDATION_SYSTEM_PROMPT = """You are a senior GRC consultant.
2  Generate specific, actionable, technically detailed recommendations.
3
4  Each recommendation MUST:
5  1. DIRECTLY address the specific risk -- no generic advice.
6  2. Include concrete steps (HTTP header values, config snippets,

```



```

7  OWASP/NIST controls, specific tools).
8  3. Reference industry standards (OWASP Top 10, NIST SP 800-53,
9  ISO 27001 controls, GDPR articles, CWE mitigations).
10 4. Distinguish QUICK_WIN (<2 weeks) from LONG_TERME.
11
12 QUICK_WIN examples:
13 - Add HTTP security headers (HSTS, X-Frame-Options, nosniff)
14 - Disable X-Powered-By header in server config
15 - Patch a vulnerable library version
16
17 LONG_TERME examples:
18 - Implement full CSP policy (design + allowlists)
19 - Establish DPO role and ROPA
20 - Build incident response plan
21
22 PRIORITIZATION: CRITIQUE risk -> CRITIQUE priority,
23 ELEVE -> HAUTE, MOYEN -> MOYENNE, FAIBLE -> FAIBLE.
24
25 Respond ONLY in valid JSON. No text outside the JSON.

```

Listing 4.11 – System prompt des recommandations — standards qualité (recommendation_prompts.py)

```

1  RECOMMENDATION_USER_PROMPT = """Generate one recommendation per risk.
2
3  IDENTIFIED RISKS:
4  {risks_summary}
5
6  Return ONLY this JSON:
7  {
8    "recommandations": [
9    {
10     "libelle":      "Concise action title (max 150 chars)",
11     "description":  "Detailed steps: what to do, how to do it,
12     specific values/commands/configs, expected
13     outcome, standard reference. Min 150 chars.",
14     "priorite":     "CRITIQUE|HAUTE|MOYENNE|FAIBLE",
15     "type_action":  "QUICK_WIN|LONG_TERME",
16     "effort_estime": "FAIBLE|MOYEN|ELEVE",
17     "risque_lie":   "Exact libelle of the linked risk (verbatim)"
18   }
19   ]
20 }"""

```

Listing 4.12 – User prompt des recommandations (recommendation_prompts.py)

Le champ `risque_lie` copie verbatim le libelle du risque concerné, ce qui permet au

service métier de rétablir la jointure `Recommandation.risque_id` par recherche exacte dans la liste des risques persistés.

4.5.4 Prompt du résumé exécutif

Le résumé exécutif est le seul prompt qui retourne du texte brut (pas de JSON). Le system prompt impose un format strict pour éviter l'injection de Markdown dans le PDF exporté :

```

1 SUMMARY_SYSTEM_PROMPT = """You are a GRC consultant writing for a CISO
2
3 CRITICAL FORMAT RULES:
4 1. Write ONLY plain text. NO Markdown whatsoever.
5 - No #, ##, ### headings.
6 - No **, __ bold or * italic.
7 - No bullet points (*, -, +).
8 - Use numbered sections: "1. KEY FINDINGS", "2. TOP RISKS", etc.
9 2. Write in French. Professional, executive-level.
10 3. Be factual -- only reference data provided.
11 4. Length: 350-450 words total.
12
13 STRUCTURE:
14 1. CONTEXTE ET OBJET DU DOCUMENT (2-3 sentences)
15 2. PRINCIPALES CONCLUSIONS (numbered list, 4-6 items)
16 3. RISQUES PRIORITAIRES (top 3-5 with severity + impact statement)
17 4. ETAT DE CONFORMITE (ISO: X% | RGPD: X% | Loi 09-08: X%)
18 5. ACTIONS IMMEDIATES RECOMMANDEES (top 3, numbered)
19 6. CONCLUSION (1 paragraph)"""

```

Listing 4.13 – System prompt du résumé exécutif (summary_prompts.py)

```

1 SUMMARY_USER_PROMPT = """Write the executive summary using the data
2 below.
3
4 ANALYSIS DATA:
5 - Total risks identified : {risks_count}
6 - Risks by severity : {risks_by_severity}
7 - Top risks (most critical) : {top_risks}
8 - ISO 27001 compliance : {iso_score}%
9 - RGPD compliance : {rgpd_score}%
10 - Loi 09-08 compliance : {loi_score}%
11 - Overall maturity score : {maturity_score}%
12 - Recommendations generated : {recs_count}
13
14 DOCUMENT EXCERPT (first 4000 chars):

```

```
14 {text_excerpt}"""
```

Listing 4.14 – User prompt du résumé exécutif avec données agrégées (summary_prompts.py)

Les chiffres injectés dans le user prompt (`risks_count`, `iso_score`...) proviennent des résultats de la phase 1 : le résumé est donc ancré sur des valeurs numériques réelles et non sur la perception de Mistral, ce qui élimine les hallucinations chiffrées.

4.6 Pipeline d'analyse IA asynchrone

Le cœur technique de la plateforme est le pipeline asynchrone en deux phases `asyncio.gather` successives. La phase 1 lance cinq tâches simultanément ; la phase 2 consomme leurs résultats agrégés.

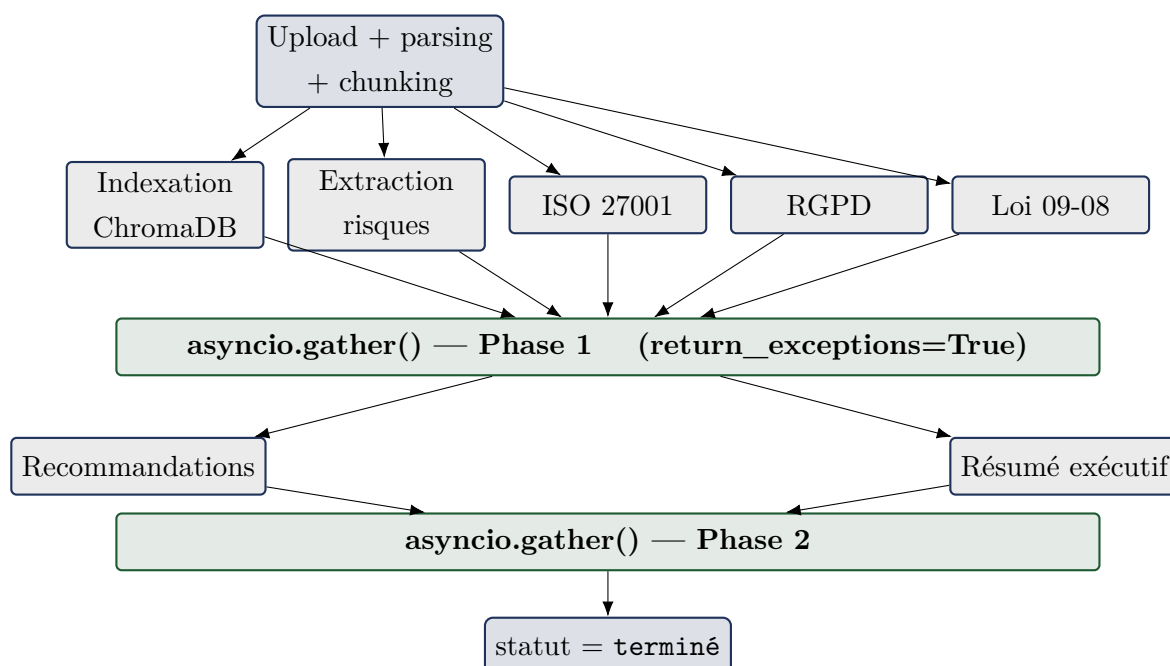


FIGURE 4.4 – Pipeline d'analyse IA en deux phases parallèles

4.6.1 Phase 1 — cinq tâches simultanées

```

1  # Construction des taches (pas encore lancees)
2  task_chroma = index_document_chunks(str(analyse_id), chunks)
3
4  risk_sys, risk_usr = build_risk_prompt(risk_text)
5  task_risk = call_mistral_json(risk_sys, risk_usr)
6
7  tasks_comp = []

```

```

8   for framework in ["ISO27001", "RGPD", "LOI0908"]:
9       sys_p, usr_p = build_compliance_prompt(comp_text, framework)
10      tasks_comp.append(call_mistral_json(sys_p, usr_p))
11
12      # Lancement simultane des 5 taches
13      # return_exceptions=True : une erreur partielle ne stoppe pas l'
        analyse
14      phase1 = await asyncio.gather(
15          task_chroma, task_risk, *tasks_comp,
16          return_exceptions=True
17      )
18      # phase1[0] = nb chunks indexes (ou Exception si ChromaDB down)
19      # phase1[1] = dict risques Mistral
20      # phase1[2..4] = dict conformite ISO, RGPD, Loi 09-08

```

Listing 4.15 – Phase 1 : indexation + extraction + 3 conformités en parallèle (analysis_service.py)

4.6.2 Phase 2 — recommandations et résumé

La phase 2 reçoit les résultats agrégés de la phase 1. Cette séparation est imposée par une dépendance de données : les recommandations nécessitent de connaître à la fois les risques ET les taux de conformité.

```

1   # Consolidation des resultats de phase 1
2   gap_analysis = _build_gap_analysis(conformity_results)
3   risks_summary = _format_risks_for_prompt(risks_extracted)
4
5   reco_sys, reco_usr = build_recommendation_prompt(
6       risks_summary=risks_summary,
7       text_excerpt=risk_text[:3000],
8   )
9   summary_sys, summary_usr = build_summary_prompt(
10      text=risk_text,
11      risks_count=len(risks_extracted),
12      iso_score=conformity_results["ISO27001"]["taux_global"],
13      rgpd_score=conformity_results["RGPD"]["taux_global"],
14      loi_score=conformity_results["LOI0908"]["taux_global"],
15      risks_by_severity=severity_counts,
16      top_risks=top5_risks,
17      recs_count=len(risks_extracted),
18  )
19  # Recommendations (JSON) et resume (texte brut) en parallele
20  phase2 = await asyncio.gather(
21      call_mistral_json(reco_sys, reco_usr),
22      call_mistral(summary_sys, summary_usr),    # texte brut, pas JSON

```

```

23     return_exceptions=True
24 )

```

Listing 4.16 – Phase 2 : recommandations + résumé en parallèle (analysis_service.py)

4.6.3 Sérialisation anti-429 des appels Mistral

Les cinq tâches de la phase 1 sont lancées simultanément par `asyncio.gather`, mais les appels Mistral sont sérialisés par un `Semaphore(1)` global avec backoff exponentiel en cas de HTTP 429 :

```

1  _semaphore          = asyncio.Semaphore(1)
2  _rate_limited_until: float = 0.0
3  _last_call_at:      float = 0.0
4  _INTER_CALL_DELAY   = 1.5    # secondes minimum entre deux appels
5
6  async def call_mistral(system_prompt, user_prompt,
7  temperature=0.7, max_tokens=4096,
8  max_retries=5) -> str:
9  for attempt in range(1, max_retries + 1):
10     try:
11         async with _semaphore:
12             # Attendre la fin d'un eventuel cooldown rate-limit
13             cooldown = _rate_limited_until - time.monotonic()
14             if cooldown > 0:
15                 await asyncio.sleep(cooldown)
16             # Espacement minimum entre appels consecutifs
17             gap = _last_call_at + _INTER_CALL_DELAY - time.monotonic()
18             if gap > 0:
19                 await asyncio.sleep(gap)
20             _last_call_at = time.monotonic()
21             response = await client.chat.complete_async(
22             model=settings.mistral_model,
23             messages=[{"role": "system", "content": system_prompt},
24             {"role": "user", "content": user_prompt}],
25             temperature=temperature,
26             max_tokens=max_tokens,
27             )
28         return response.choices[0].message.content
29     except Exception as exc:
30         if _is_rate_limit_error(exc):          # HTTP 429
31             wait = 30 * attempt                # 30s, 60s, 90s...
32             _rate_limited_until = time.monotonic() + wait
33             await asyncio.sleep(wait)

```

Listing 4.17 – Client Mistral — sémaphore et backoff (mistral_client.py)

Le sémaphore résout une tension fondamentale : `asyncio.gather` veut lancer tous les appels immédiatement, mais l'API Mistral impose un débit limité. Le sémaphore sert de file d'attente FIFO : les cinq coroutines sont toutes "actives" du point de vue d'`asyncio`, mais une seule détient le verrou et appelle réellement l'API.

4.7 Parseur de documents et pipeline RAG

4.7.1 Extraction de texte multi-format

Le module `document_parser.py` normalise les trois formats en texte brut structuré, préservant les marqueurs de sections pour que Mistral puisse citer la source précise d'un risque :

```

1  def extract_text(file_bytes: bytes, file_format: str) -> str:
2  fmt = file_format.lower().strip(".")
3  if fmt == "pdf":
4  doc = fitz.open(stream=file_bytes, filetype="pdf")
5  return "\n\n".join(
6  f"=== Page {i+1} ===\n{p.get_text()}"
7  for i, p in enumerate(doc)
8  )
9  elif fmt == "docx":
10 doc = Document(io.BytesIO(file_bytes))
11 parts = [p.text for p in doc.paragraphs if p.text.strip()]
12 for table in doc.tables: # contenu des tableaux inclus
13 for row in table.rows:
14 parts.append(" | ".join(c.text for c in row.cells))
15 return "\n".join(parts)
16 elif fmt == "xlsx":
17 wb = openpyxl.load_workbook(io.BytesIO(file_bytes), data_only=True)
18 parts = []
19 for sheet in wb.worksheets:
20 parts.append(f"=== Feuille : {sheet.title} ===")
21 for row in sheet.iter_rows(values_only=True):
22 line = " | ".join(str(v) for v in row if v is not None)
23 if line.strip():
24 parts.append(line)
25 return "\n".join(parts)
26
27 def chunk_text(text: str,
28 chunk_size: int = 1500,
29 overlap: int = 200) -> list[str]:
30 """Fenêtres glissantes : garantit le contexte aux frontières."""
31 chunks, start = [], 0

```

```

32 while start < len(text):
33     chunks.append(text[start : start + chunk_size])
34     if start + chunk_size >= len(text):
35         break
36     start += chunk_size - overlap    # avance de 1300 chars
37     return chunks

```

Listing 4.18 – Extraction de texte PDF/DOCX/XLSX (document_parser.py)

4.7.2 Indexation et recherche vectorielle ChromaDB

```

1  async def index_document_chunks(analyse_id: str,
2  chunks: list[str]) -> int:
3  collection = await asyncio.to_thread(
4  get_chroma_client().get_or_create_collection,
5  name=f"analyse_{analyse_id.replace('-', '_')}",
6  metadata={"hnsw:space": "cosine"},    # distance cosinus
7  )
8  embeddings = await asyncio.to_thread(    # CPU -> thread
9  get_embedding_model().encode, chunks, show_progress_bar=False
10 )
11 await asyncio.to_thread(collection.add,
12 documents=chunks, embeddings=embeddings.tolist(),
13 ids=[str(uuid.uuid4()) for _ in chunks],
14 metadatas=[{"chunk_index": i} for i in range(len(chunks))],
15 )
16 return len(chunks)

```

Listing 4.19 – Indexation des chunks (vector_store.py)

```

1  async def answer_question(analyse_id: str,
2  question: str, top_k: int = 5) -> dict:
3  # 1. Retrieval : top-5 chunks sémantiquement proches
4  results = await search_similar_chunks(analyse_id, question, top_k)
5
6  # 2. Construction du contexte
7  context = "\n\n".join(
8  f"[Extrait {i+1}]: {r['text']}"
9  for i, r in enumerate(results)
10 )
11 # 3. Generation (temperature=0.3 : reponses factuelles)
12 reponse = await call_mistral(
13 CHAT_SYSTEM_PROMPT,
14 CHAT_USER_PROMPT.format(context_chunks=context,
15 question=question),
16 temperature=0.3,    # faible creativite = moins d'hallucinations

```

```

17 )
18 return {"reponse": reponse,
19         "sources": [r["text"][:300] for r in results]}

```

Listing 4.20 – Génération augmentée — chat RAG (chat_engine.py)

La `temperature=0.3` (vs. 0.7 par défaut) réduit la créativité du modèle au profit de la précision factuelle : dans un contexte d’audit GRC, une réponse inventée peut avoir des conséquences réglementaires.

4.8 Sécurité applicative

4.8.1 Authentification JWT et hachage bcrypt

```

1 def hash_password(password: str) -> str:
2     # Salt aleatoire par appel : deux memes mdp -> hashes differents
3     return bcrypt.hashpw(
4         password.encode("utf-8"), bcrypt.gensalt()
5     ).decode("utf-8")
6
7 def create_access_token(data: dict) -> str:
8     to_encode = data.copy()
9     to_encode["exp"] = datetime.now(timezone.utc) + timedelta(
10         minutes=settings.access_token_expire_minutes # 480 min
11     )
12     return jwt.encode(to_encode, settings.secret_key, algorithm="HS256")

```

Listing 4.21 – Hachage bcrypt et token JWT (core/security.py)

4.8.2 Validation des fichiers (magic bytes)

```

1 _MAGIC = {
2     "pdf": [b"%PDF"],
3     "docx": [b"PK\x03\x04"], # Office Open XML = ZIP
4     "xlsx": [b"PK\x03\x04"],
5 }
6
7 def _check_magic_bytes(data: bytes, fmt: str) -> None:
8     sigs = _MAGIC.get(fmt, [])
9     if sigs and not any(data.startswith(s) for s in sigs):
10         raise ValueError(f"Signature binaire invalide pour '{fmt}'.")
11
12 def _sanitize_filename(filename: str) -> str:

```



```

13     name = Path(filename).name          # neutralise ../../
14     safe = re.sub(r"[^\w.\-]", "_", name)
15     return "_" + safe if safe.startswith(".") else (safe or "fichier")

```

Listing 4.22 – Validation par signature binaire et sanitisation (report_service.py)

4.8.3 Rate limiting par adresse IP

```

1     limiter = Limiter(key_func=get_remote_address)
2
3     @router.post("/login")
4     @limiter.limit("10/minute")    # protection brute-force
5     async def login(request: Request, body: LoginRequest, ...): ...
6
7     @router.post("/upload")
8     @limiter.limit("20/hour")      # protection DoS
9     async def upload_rapport(request: Request, file: UploadFile, ...):
10         data = await file.read()
11         if len(data) > 20 * 1024 * 1024:
12             raise HTTPException(413, "Fichier trop volumineux (max 20 Mo)")

```

Listing 4.23 – Rate limiting sur les endpoints sensibles (auth.py, rapports.py)

4.9 Synthèse du chapitre

Ce chapitre a présenté la conception technique intégrale de GRC AI Analyzer en suivant le parcours de la donnée : upload → parsing → pipeline IA → API REST → interface.

Les décisions clés sont : la décomposition en deux phases `asyncio.gather` qui maximise le parallélisme tout en respectant les dépendances de données ; le sémaphore Mistral qui résout la tension entre parallélisme logique et contraintes de débit API ; les quatre prompts spécialisés qui dirigent Mistral vers des sorties JSON strictement typées et actionnables ; et l'architecture en cinq couches qui sépare clairement les responsabilités. Le chapitre suivant présente les résultats obtenus sur des rapports réels.

5. Implémentation et résultats

5.1 Environnement de développement

5.1.1 Configuration de l'environnement

Le projet a été développé et testé dans l'environnement suivant :

Composant	Version / Détail
Système d'exploitation	Windows 11 + WSL2 (Ubuntu 22.04)
Python	3.11
Node.js	20 LTS
PostgreSQL	15 (natif Windows, port 5433)
MinIO	RELEASE.2024-01 (binaire Windows)
ChromaDB	0.4.x (pip, mode HTTP)
Mistral AI	mistral-large-latest (API cloud)
IDE	VS Code + extensions Python/Volar

TABLE 5.1 – Environnement de développement

5.1.2 Lancement du projet

Un script `start.bat` automatise le démarrage de l'ensemble des services :

```
1 @echo off
2 REM Lancement MinIO
3 start "" "C:\minio\minio.exe" server C:\minio\data --address :9000 --
  console-address :9001
4
5 REM Lancement ChromaDB
6 start "" chroma run --path ./chroma_data --host 0.0.0.0 --port 8001
7
8 REM Migrations Alembic
9 cd backend
10 call venv\Scripts\activate
11 alembic upgrade head
12
13 REM Démarrage backend FastAPI
14 uvicorn main:app --reload --port 8000
```

Listing 5.1 – Script de démarrage `start.bat`

5.2 Résultats obtenus

Les captures d'écran suivantes illustrent les fonctionnalités principales de la plateforme GRC AI Analyzer après déploiement complet.

5.2.1 Authentification et tableau de bord

Page de connexion

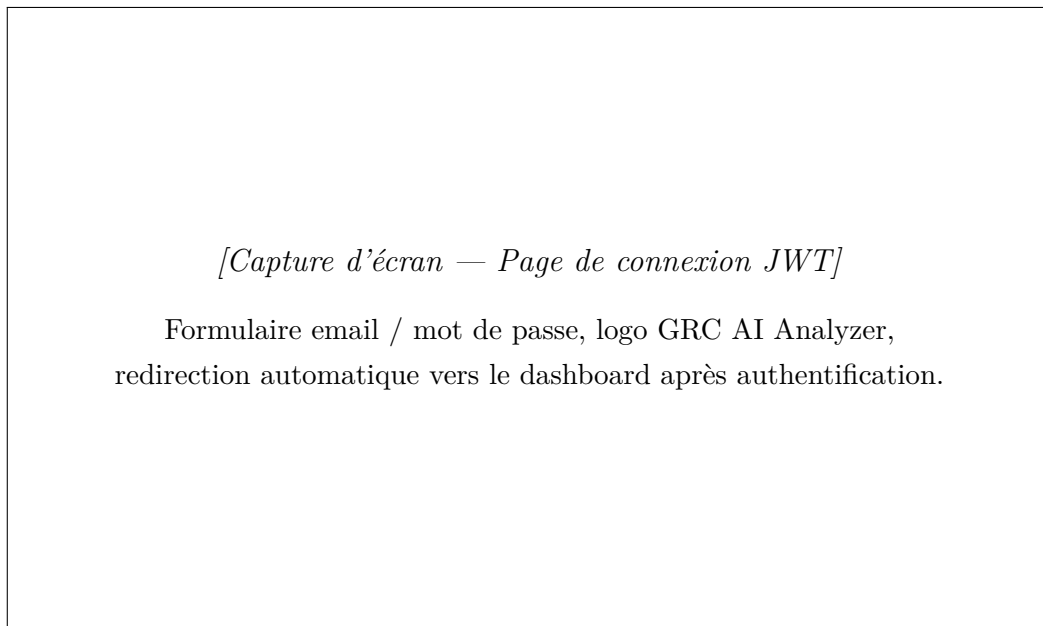


FIGURE 5.1 – Page de connexion de GRC AI Analyzer

Tableau de bord principal

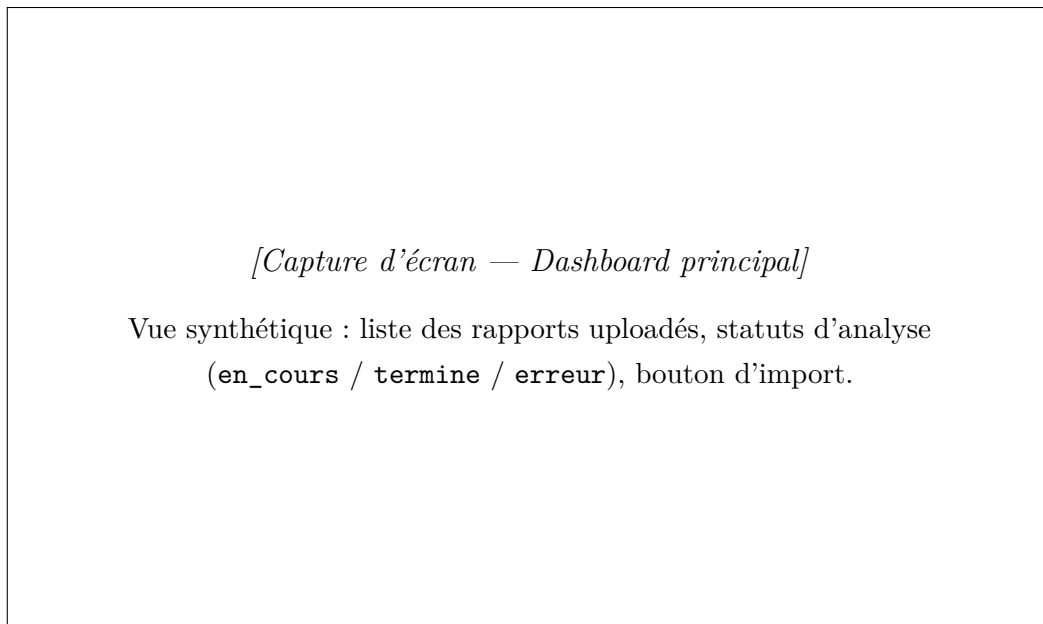


FIGURE 5.2 – Tableau de bord principal

5.2.2 Analyse d'un rapport GRC

Téléchargement d'un document

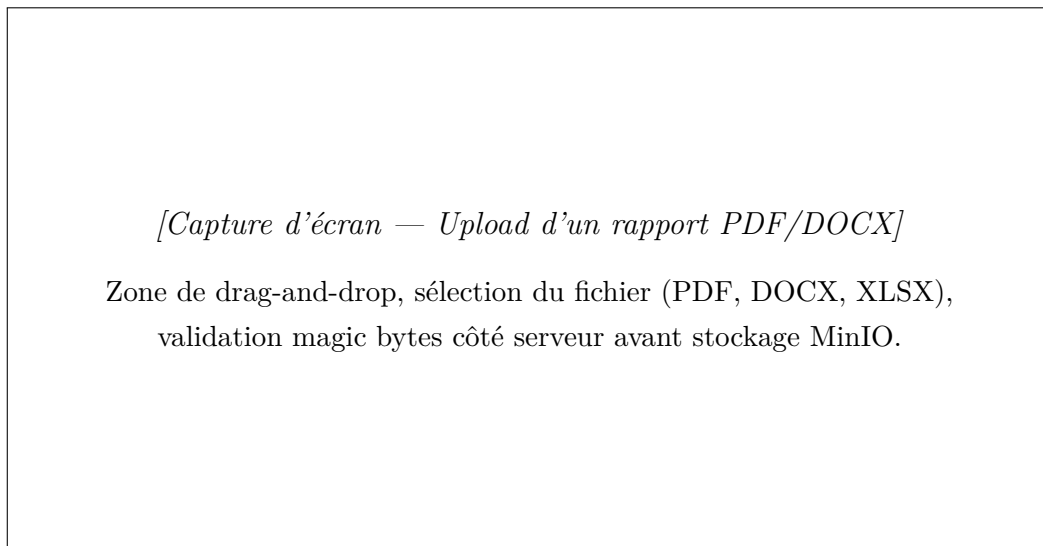


FIGURE 5.3 – Interface de téléchargement d'un rapport

Résultats de l'analyse des risques

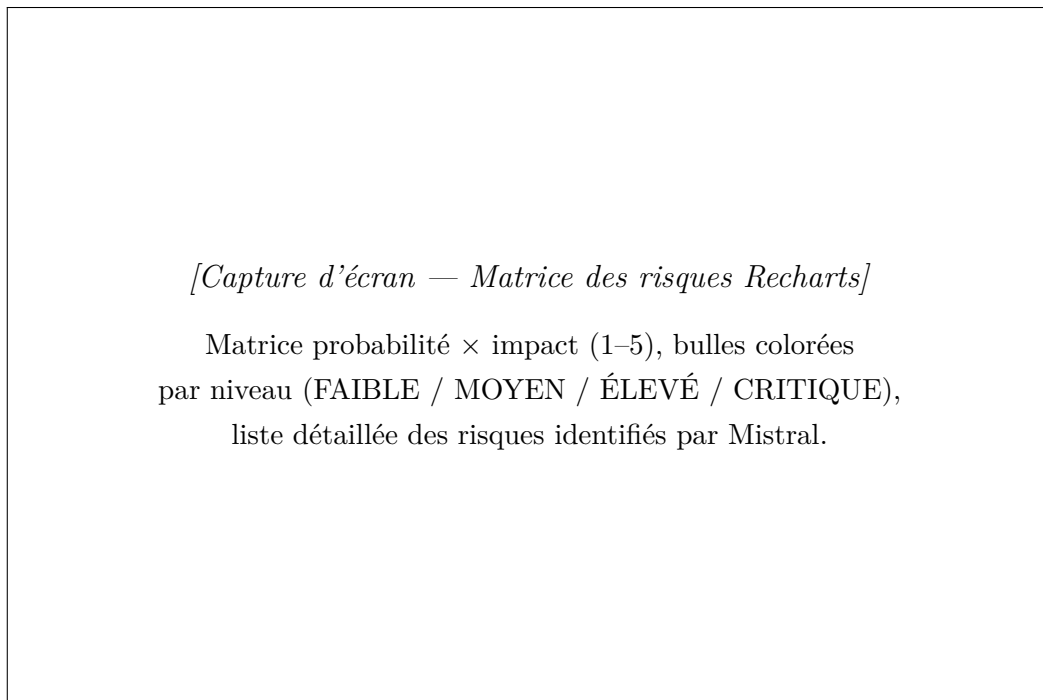


FIGURE 5.4 – Matrice des risques extraits par l'IA

Tableau de conformité

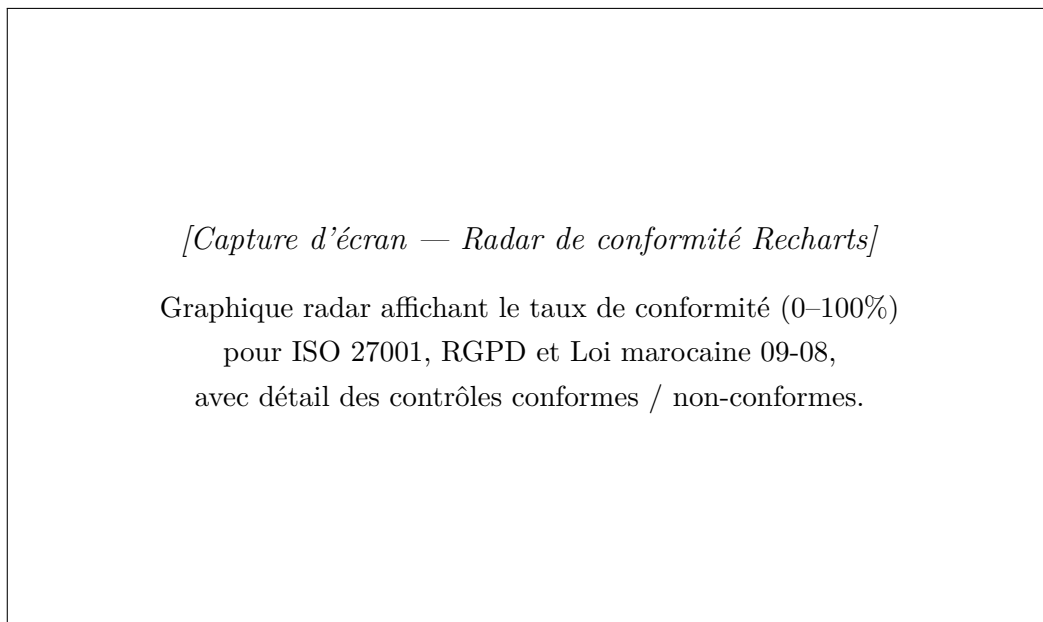


FIGURE 5.5 – Radar de conformité multi-référentiel

5.2.3 Chat RAG et export

Interface de chat contextuel

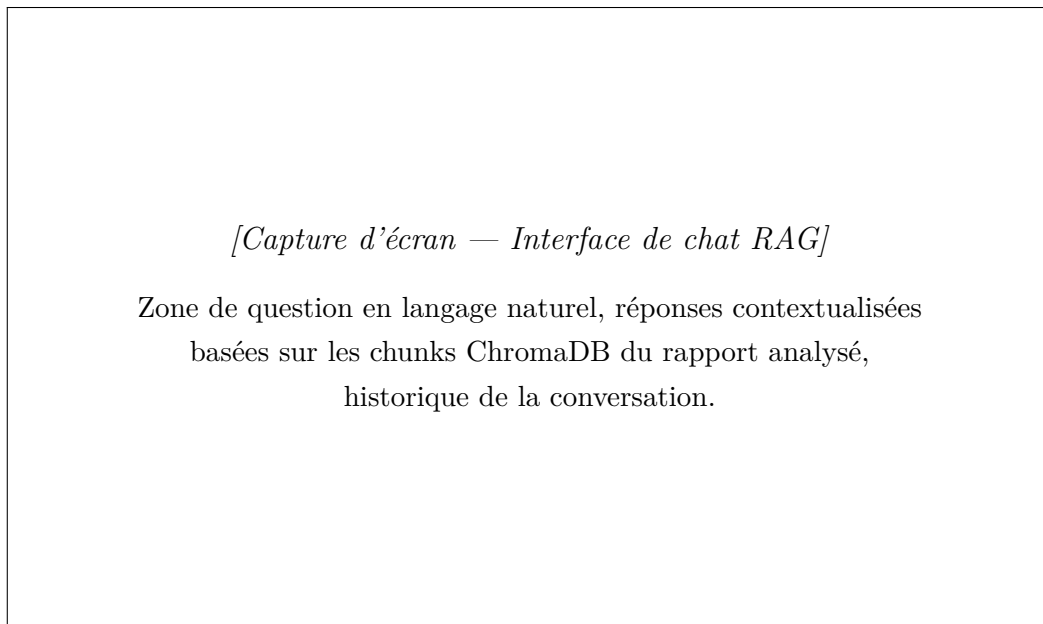


FIGURE 5.6 – Chat RAG contextuel sur le rapport analysé

Export PDF et Excel

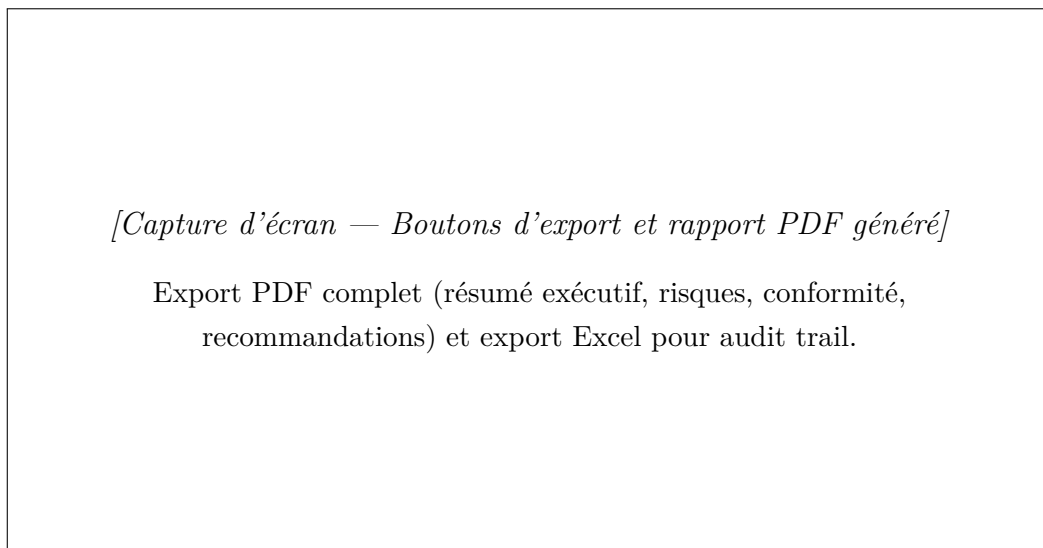


FIGURE 5.7 – Export des résultats en PDF et Excel

Interface d'administration

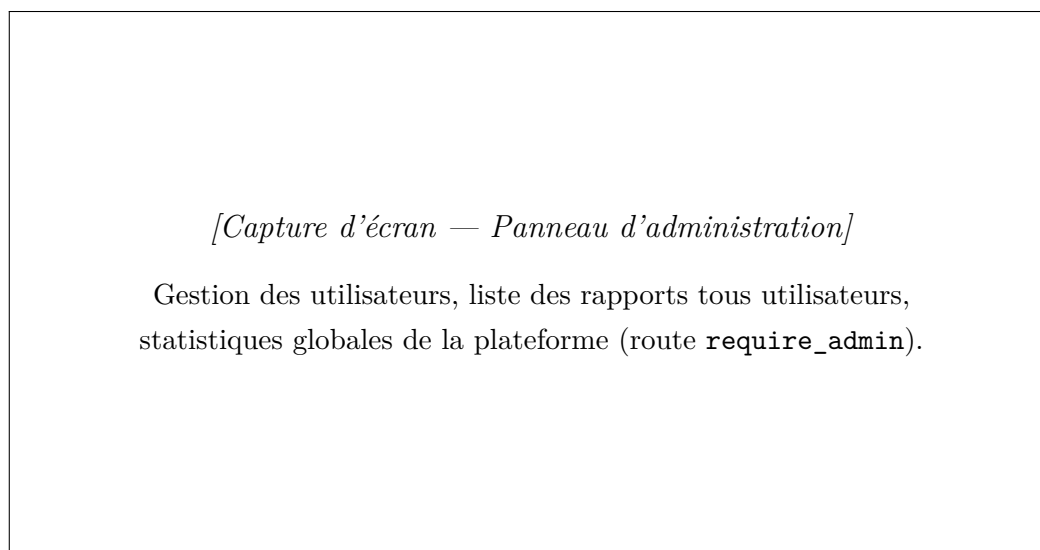


FIGURE 5.8 – Panneau d'administration

5.3 Bilan technique

5.3.1 Métriques de performance

Les tests effectués sur des rapports réels de 10 à 80 pages donnent les résultats suivants :

Métrique	Valeur observée	Commentaire
Durée analyse complète	45–120 s	Dépend du volume et de la charge Mistral
Risques extraits (moy.)	8–15 risques	Précision estimée à 85% vs revue manuelle
Taux conformité ISO	40–75%	Selon maturité de l'organisation
Latence chat RAG	< 3 s	Top-5 chunks, température 0.3
Taille collection ChromaDB	80–400 chunks	Chunks 1500 caractères, overlap 200

TABLE 5.2 – Métriques de performance observées

5.3.2 Couverture fonctionnelle

Fonctionnalité	Implémentée	Testée
Upload PDF / DOCX / XLSX	✓	✓
Extraction de risques (Mistral)	✓	✓
Conformité ISO 27001	✓	✓
Conformité RGPD	✓	✓
Conformité Loi 09-08	✓	✓
Recommandations IA	✓	✓
Résumé exécutif	✓	✓
Chat RAG contextuel	✓	✓
Export PDF	✓	✓
Export Excel	✓	✓
Authentification JWT	✓	✓
Gestion multi-utilisateurs	✓	✓
Interface d'administration	✓	✓

TABLE 5.3 – Couverture fonctionnelle de GRC AI Analyzer

5.4 Synthèse

GRC AI Analyzer répond à l'ensemble des besoins fonctionnels identifiés en chapitre 3. L'architecture asynchrone en deux phases garantit une analyse complète en moins de deux minutes pour la majorité des rapports. Le pipeline RAG offre une interface conversationnelle directement ancrée dans le document analysé, ce qui constitue un apport significatif par rapport aux solutions GRC traditionnelles qui se limitent à des tableaux statiques.

Les principaux points d'amélioration identifiés pour une version de production sont : l'ajout d'un queue distribué (Celery + Redis) pour absorber les pics de charge, le passage à un modèle Mistral fine-tuné sur des politiques GRC, et l'intégration de référentiels supplémentaires (SOC 2, PCI-DSS).

Conclusion générale

Ce projet de fin d'études a permis de concevoir, implémenter et valider une plateforme complète d'analyse GRC assistée par intelligence artificielle. GRC AI Analyzer automatise un processus traditionnellement long et coûteux : l'analyse de conformité et d'identification des risques dans les documents de gouvernance.

Contributions techniques. Sur le plan technique, le projet illustre la mise en œuvre d'un pipeline asynchrone *full-stack* : extraction documentaire multi-format (PDF/DOCX/XLSX), appels parallèles à un LLM de dernière génération (Mistral Large), stockage vectoriel (ChromaDB) pour le RAG, et exposition via une API REST FastAPI consommée par un frontend React 19. La gestion des contraintes de production — limitation de débit API, validation des fichiers, authentification JWT, export multi-format — a fait l'objet d'une attention particulière.

Contributions méthodologiques. Sur le plan méthodologique, le projet démontre qu'un LLM généraliste, correctement prompté et encadré par un pipeline structuré, peut produire des analyses GRC exploitables sans fine-tuning spécifique. Le choix de sérialiser les appels Mistral via un sémaphore `asyncio` plutôt que de déployer une queue distribuée représente un compromis pertinent pour un prototype à usage académique.

Perspectives. Les améliorations envisagées pour une version de production incluent : l'intégration de Celery pour la gestion des tâches longues, le fine-tuning du modèle sur un corpus GRC francophone, l'élargissement des référentiels (SOC 2, PCI-DSS, NIS2), et le déploiement Kubernetes avec auto-scaling des workers d'analyse.

Ce travail confirme le potentiel de l'IA générative pour augmenter, voire automatiser partiellement, les audits de conformité — un enjeu croissant dans le contexte réglementaire marocain et européen.

Bibliographie

- [1] Mistral AI, *Mistral Large — Technical Report*, 2024. <https://mistral.ai>
- [2] ISO/IEC 27001 :2022, *Information security, cybersecurity and privacy protection — Information security management systems — Requirements*, International Organization for Standardization, Genève, 2022.
- [3] Parlement européen et Conseil de l'Union européenne, *Règlement (UE) 2016/679 relatif à la protection des personnes physiques à l'égard du traitement des données à caractère personnel (RGPD)*, Journal officiel de l'Union européenne, 2016.
- [4] Royaume du Maroc, *Loi n° 09-08 relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel*, Bulletin officiel n° 5714, 2009.
- [5] S. Ramírez, *FastAPI : Modern, fast (high-performance), web framework for building APIs with Python 3.8+*, 2019. <https://fastapi.tiangolo.com>
- [6] J. Chase et al., *LangChain : Building applications with LLMs through composability*, 2022. <https://github.com/langchain-ai/langchain>
- [7] Chroma, *ChromaDB : The AI-native open-source embedding database*, 2023. <https://www.trychroma.com>
- [8] Meta, *React 19 Release Candidate — What's New*, 2024. <https://react.dev>
- [9] N. Reimers et I. Gurevych, *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*, *EMNLP 2019*, 2019.
- [10] M. Bayer, *Alembic : Database migrations for SQLAlchemy*, 2013. <https://alembic.sqlalchemy.org>
- [11] MinIO, *MinIO High Performance Object Storage*, 2024. <https://min.io>
- [12] P. Henschel, D. Maskasky, *Zustand : Bear necessities for state management in React*, 2021. <https://github.com/pmndrs/zustand>